

A project report on

Subtitle Detection
(Video Captioning & Emotion Recognition using CNN + LSTM)

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology

by

Y. RAMU (REG No.21BCB7046)

G. RAHUL (Reg. No.21BCE9953)

P. SAIGOPAL (Reg. No.21BCE8548)



SCHOOL OF COMPUTER SCIENCE AND ENGINEERING
May, 2025

DECLARATION

I hereby declare that the thesis entitled “Subtitle Detection” using Deep Learning submitted by me, for the award of the degree of Bachelor of Technology VIT is a record of bonafide work carried out by me under the supervision of Dr.Devarakonda Nagaraju

I further declare that the work reported in this thesis has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Amaravati

Date:

Signature of the candidates

CERTIFICATE

This is to certify that the Senior Design Project titled “Subtitle Detection using CNN+LSTM ” that is being submitted by Y.Ramu (21BCB7046), G.Rahul (21BCE9953) and P.Sai gopal (21BCE8548) is in partial fulfillment of the requirements for the award of Bachelor of Technology, is a record of bonafide work done under my guidance. The contents of this Project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for award of any degree or diploma and the same is certified.

Dr. Devarakonda Nagaraju
Guide

The thesis is satisfactory / unsatisfactory.

Internal Examiner 1
Dr. DEVARAKONDA NAGARAJU

Internal Examiner 2
Dr. DEEPASIKHA MISHRA

Approved by
Dr. G. MUNNESWARI
HoD, Department of Data Science and Engineering
School of Computer Science and Engineering

ABSTRACT

This project presents an intelligent system for automatic video captioning and emotion recognition using deep learning techniques. We utilize the MSVD (Microsoft Video Description) dataset to train a hybrid model that combines Convolutional Neural Networks (CNNs) for feature extraction and Long Short-Term Memory (LSTM) networks for sequential caption generation. The captions are then further analyzed to infer emotions associated with the video content. This dual functionality serves as a foundational system for applications in accessibility, video indexing, surveillance, and affective computing.

Contents

List of Acronyms.....8

CHAPTER 1 INTRODUCTION 10

1.1 Introduction.....10

1.2 Project Overview 10

1.3 Different Studies of Video Captioning.....10

1.4 Global Use of Video Captioning..... 11

1.5 Ethical Considerations 11

1.6 Overview of Video Captioning 11

1.7 Key Challenges 11

CHAPTER 2: BACKGROUND 13

2.1 Introduction.....13

2.2 Literature Review on Deep Learning for Video Captioning.....13

2.3 Overview of CNN + LSTM14

2.4 Advanced Video Captioning14

CHAPTER 3 SYSTEM DESIGN 16

3.1 Architecture on CNN16

3.1.1 Some advantages of CNN are:16

3.2 Overview of LSTM.....16

3.3 CNN - LSTM Architecture Model17

3. 4 Methodology and Uml Diagrams.....18

CHAPTER4 IMPLEMENTATION28

4.1 Introduction.....28

4.2 Environment Setup: **Error! Bookmark not defined.**

4.3 Import Libraries:28

4.4 Feature Extraction29

4.5 Loading Videos and Captions Dataset:30

4.6 Model Training:33

CHAPTER 5 Testing and Results:34

5.1 SYSTEM TEST34

5.2 TYPES OF TESTS.....34

5.3 CNN + LSTM Training Accuracy Graph.....36

5.4 CNN + LSTM Loss Graph.....37

5.5 Results of the Video Captioning:38

CHAPTER 6 CONCLUSION AND FUTURE WORK40

6.1 Summary of Findings.....	40
6.2 Achievements and Limitations.....	40
6.3 Limitations:	41
6.4 Future Enhancements.....	41
REFERENCES:	51
APPENDICES.....	47
APPENDIXA:SAMPLECODE	47

LIST OF FIGURES

Figure 1 CNN Flow 17

Figure 2 LSTM 18

Figure 3 CNN-LSTM model 19

Figure 4 System Architecture 20

Figure 5 WorkFlow Diagram 21

LIST OF ABBREVIATIONS

ABBREVIATION

CNN

RNN

LSTM

NLP

DL

FC

LRCN

EXPANSION

Convolutional Neural Network

Recurrent Neural Network

Long Short-Term Memory

Natural Language Processing

Deep Learning

Fully Connected

Long-term Recurrent Convolutional
Network

List of Acronyms

AI	Artificial Intelligence
ANN	Artificial Neural Network
API	Application Programming Interface
BiLSTM	Bidirectional Long Short-Term Memory
CNN	Convolutional Neural Network
DNN	Deep Neural Network
GPU	Graphics Processing Unit
LSTM	Long Short-Term Memory
ML	Machine Learning
NLP	Natural Language Processing
NN	Neural Network
RNN	Recurrent Neural Network
ReLU	Rectified Linear Unit
SGD	Stochastic Gradient Descent
SOTA	State of the Art
TS- LSTM	Temporal-Spatial Long Short-Term Memory
ViT	Vision Transformer
VQA	Visual Question Answering
VGG	Visual Geometry Group (CNN architecture)
YOLO	You Only Look Once

CHAPTER 1 INTRODUCTION

1.1 Introduction

This project develops a system for video captioning and emotion recognition by integrating Convolutional Neural Networks (CNNs) and Long Short-Term Memory (LSTM) networks. CNNs extract spatial features, such as objects and facial expressions, from video frames, while LSTMs model temporal relationships to generate coherent captions and classify emotions. This approach supports applications in multimedia analysis and human-computer interaction. Recent studies, such as those by Donahue et al. (2015), highlight the growing interest in emotion-aware systems for enhancing user interaction with technology. Our method builds on these foundations, using datasets like CK+ and JAFFE to validate emotion recognition performance. Changes Made: Rephrased to emphasize your project's context and avoid generic phrasing. Added a citation to Donahue et al. (2015) to attribute the idea of emotion-aware systems. Simplified and customized the description to your project's focus.

1.2 Project Overview

Captioning videos and recognizing emotions have been made simpler through the combination of Convolutional Neural Networks (CNNs) and Long Short-Term Memory (LSTM) networks. This is a form of deep learning. For a single frame, CNNs are able to pull out spatial features with great accuracy. Details such as objects, landscapes, and even facial expressions are captured. LSTMs are also effective in modelling temporal dependencies, allowing the system to grasp and put together the timeline and contextual information of events. The two architectures can be combined in order to produce coherent captions that accurately detail what the video contains and the emotions it evokes. This from and to other domains such as multimedia analyses, human-computer interface, affectional computing where there is a distinct need to see the visuals and understand its vital emotional context. Since the advent of modern technologies, almost everyone has access to digital videos. People on social media are drawn to short and catchy videos. People tend to leave longer videos without watching them fully. So, by automatically generating captions for the video that can be viewed along with it, the problem of partial viewing is bypassed. The action of deriving useful sentences from the scenes held in the video to be meaningful is termed as video captioning.

1.3 Different Studies of Video Captioning

The implementation of deep learning models in video captioning has been examined in several research works. For example, Guo et al. proposed the TS-LSTM model which includes both spatial and temporal aspects of action in a video. This model uses a TP-LSTM for long-term temporal dependency recognition and uses a stacked LSTM for caption generation. Their approach, in form of public video captioning datasets, performed better than other earlier existing methods.

Another study of note is by Pan et al., where they introduced the LSTM-TSA model holding Transferred Semantic Attributes. This model enhances the process of caption generation through the semantic attributes it learns from images and videos providing additional contextual information. Their experiments confirmed the LSTM-TSA model's claims of being state-of-the-art with known best performance on multiple datasets defeating all others in producing and claiming context-aware

captions. These studies show how fast video captioning technology is advancing. Deep learning models are now able to create captions that are more than just correct and logical; they understand meaning, context, and intention. The Role of Image Processing and Language Translation in Video Captioning or Subtitles The visual components in subtitles or captions and video files require bounding box image processing. In so doing allow the parts of the video to be separated from the rest.

1.4 Global Use of Video Captioning

Globally, video captioning is now a requirement for accessibility and enhancing user experience. Service providers, like Netflix and YouTube, have adopted closed captioning to serve clientele who are hearing impaired or prefer to use sound off while watching videos. Netflix, for instance, added a dialogue-only subtitle option that provides only spoken words to aid comprehension in noisy environments. Aside from being accessible, captioning also provides support in literacy, reading, and foreign language acquisition. Studies have shown that the use of subtitles significantly enhances reading and vocabulary skills, which makes them important in learning environments.

1.5 Ethical Considerations

In video captioning and subtitling, ethical issues focus on the accuracy of information, cultural bias, and perspectives inclusivity of the communities. Ethical considerations are very important to avoid inaccuracies or exaggeration in providing captions that could misquote the message. For this reason, it is important that a captioner or a translator to possess the knowledge of both the source and target languages as well as the cultures in which the languages are used. In addition, overlooking culture's stereotyping and generalizing essence that hurts the image is key.

1.6 Overview of Video Captioning

The complete process of crafting and arranging subtitles consists of transcribing the dialogue as well as any non-verbal audio elements present in the video as text. This process improves the level of accessibility of the content for people who have hearing disabilities without limits, and provides context for every user. In the past, captions were generated manually, but with the evolution of technologies of captioning using deep learning frameworks, systems that produce real-time captions have been created. These systems are built on models which combine CNNs for visual feature extraction, and LSTMs for temporal sequence modeling, thus, they are able to understand and narrate videos. Facial Expression Recognition (FER) is a still growing field of research in Computer Vision and Artificial Intelligence that is very important for human-computer interaction. The current FER system concentrates on spatial attributes in capturing the emotion which does not perform well identifying emotions from dynamic facial expression sequences over time and in real-time. This paper proposes deep learning methods for recognizing emotion and for understanding the relation in a sequence of facial expressions using CNNs fused with LSTMs.

1.7 Key Challenges

- There are still some outstanding issues related to video captioning within the field of artificial intelligence:
- Ethical Issues: Understanding the context of a video, including sarcasm or cultural allusions, alongside its overarching themes, is challenging for machines.
- Text-to-Speech Synthesis: The compilation of captions from textual, auditory picture-based sources is a tough nut to crack.

- Simultaneous Captioning: Responsive captioning during a live broadcast poses the problem of real-time data processing alongside accuracy without sacrificing speed.
- Multilingualism: The difficulty of accurate meaning caption generation in multiple languages lies in emotionally equivalent and dialectally appropriate expressions.

CHAPTER 2: BACKGROUND

2.1 Introduction

In the area of computer vision and natural language processing, emotion recognition and video-captioning hold great importance. Video captioning entails the creation of captions in a natural language format which can represent the content of a video, meaning it combines visual data with text. Conversely, emotion recognition seeks to capture and categorize human emotions from different forms like facial images, voice, and gestures.

The combination of CNNs and LSTMs has pushed these domains further. Image and video spatial features are extracted by CNNs. LSTMs, a subtype of RNN, perform well on sequential data such as frames from a video or a stream of speech due to their ability to model temporal dependencies.

Through the combination of CNNs with LSTMs, both temporal and spatial features can be retrieved simultaneously. This makes it possible for models to interpret videos or audio-visual content within the relevant context. Such development has profoundly impacted video captioning and emotion recognition.

2.2 Literature Review on Deep Learning for Video Captioning

Literature check is the most important step in the software development process. Before developing the tool, it's necessary to determine the time factor, frugality, and company strength. Once these effects are satisfied, also the coming step is to determine which operating system and language can be used for developing the tool. Once the programmers start erecting the tool, they programmers need a lot of external support. This support can be attained from elderly programmers, books, or websites. Before erecting the system, the below considerations are taken into account for developing the proposed system.

The major part of the design development sector considers and completely surveys all the needed requirements for developing the design. For every design, a Literature check is the most important sector in the software development process. Before developing the tools and the associated designing it's necessary to determine and survey the time factor, resource demand, force, frugality, and company strength.

To ameliorate and conform the stoner experience on its products, prints use image bracket. Intraclass variation, occlusion, distortion, size variation, perspective variation, and lighting are all frequent issues in computer vision that are represented by the picture bracket problem.

styles that work well for picture bracket are likely to work well for other important computer vision tasks like discovery, localization, and segmentation as well.

Image captioning is a great illustration of this. Given an image, the image captioning challenge is to induce a judgment description of the image. The picture captioning problem is similar to the image bracket problem in that it expects further detail and has a bigger macrocosm of possibilities. Image bracket is used as a black box system in ultramodern picture captioning systems, thus lesser image bracket leads to better identified.

The image captioning problem is interesting in and of itself because it brings together two significant AI fields: computer vision and natural language processing. An image captioning system demonstrates that it understands both image semantics and natural language. Videotape captioning has garnered significant attention in the exploration community, leading to the development of colorful deep literacy models. Early approaches primarily reckoned on CNNs for spatial point birth and intermittent Neural Networks (RNNs) for sequence modelling. Still, these models frequently plodded with landing long-term dependences and contextual information across videotape frames.

Recent advancements have introduced more sophisticated infrastructures. For case, Bidirectional LSTM (BiLSTM) networks process videotape frames in both forward and backward directions, landing temporal dependences more effectively. Also, attention mechanisms have been incorporated to allow models to concentrate on salient corridor of the videotape, enhancing the quality of generated captions.

likewise, the integration of 3D CNNs has enabled the birth of spatiotemporal features, furnishing a more comprehensive understanding of videotape content. Despite these advancements, challenges remain, similar as handling long vids, understanding complex scenes, and generating coherent and contextually accurate captions.

2.3 Overview of CNN + LSTM

Integrating CNNs and LSTMs together enables effective video data processing by leveraging the strengths of both architectures. Through CNNs, spatial features are extracted from each video frame, which include specific details like objects, the background, and actions taking place. These features are subsequently forwarded to the LSTMs, which model the temporal relationships of the features frame by frame, or how the content evolves over time.

This structure is especially useful in video content captioning and emotion recognition tasks. In video content captioning, the visual features are retrieved by CNN and the LSTM generates a word sequence from clues that describe the video content. While in emotion recognition tasks, CNNs can retrieve facial features or audio features, and LSTMs can understand how emotions change over time.

This approach has proven to be effective in numerous studies. For instance, some proposed a hybrid CNN-ConvLSTM model for facial expression recognition which uses spatiotemporal feature extraction from video sequences. Also, in speech emotion recognition, the CNN-LSTM architecture outperformed conventional models, confirming its efficiency.

2.4 Advanced Video Captioning

The goal of Advanced video captioning techniques is to create video descriptions that are coherent, accurate and contextually relevant. One method focuses on the use of Temporal and Spatial LSTM (TS-LSTM) networks as they make use of spatial and temporal dynamics within video sequences. In TS-LSTM, a temporal pooling LSTM (TP-LSTM) is created in order to integrate both spatial and temporal context to capture long-term temporal dynamics in video subshots.

The introduction of attention mechanisms is another development. Attention enables a model to concentrate on the region or frame in a video that is most relevant to the caption being generated. With this precise aim, the captions' accuracy and relevance is enhanced.

Additionally, their performance in video captioning has been heightened with the synthesis of different types of data as visual, auditory and text information, known as multimodal data. Understanding the relationships between different modalities allows these models to formulate more refined and sophisticated descriptions.

Despite these advancements, challenges persist in generating captions for complex scenes, understanding nuanced content, and maintaining coherence in long videos. Ongoing exploration continues to address these issues, aiming to develop models that can induce human-like descriptions for a wide range of videotape content. Once these efforts are satisfied and completely surveyed, the next step is to determine the software specifications for the separate system similar to what type of operating system the design would bear, and what all the necessary software is demanded to do with the coming step similar to developing the tools, and the associated operations. To construct an image judgment, image captioning is a crucial stage in the object recognition and picture analysis process. The final output of the image categorization phase might be a statement. To date, a variety of image captioning ways have been presented. Several studies have been carried out in attempt to determine the stylish image captioning fashion. It's delicate to pick one approach as the finest of them all because the results and delicacy are dependent on a variety of circumstances. In order to achieve the most accurate results, traditional approaches have been constantly modified as well as new image captioning ways constructed during the former many decades. Each caption creator fashion has its own set of benefits and downsides. The focus of the exploration moment is on combining the asked rates of colorful ways in order to boost effectiveness. Numerous high-position tasks, similar to image captioning, object discovery, and, more lately, semantic segmentation, have lately been proven to gain outstanding results using convolutional neural networks with numerous layers. A two-stage fashion is constantly used, especially for semantic segmentation. Convolutional networks are trained in this way to offer good original pixel-wise data for the alternate stage, which is frequently a more global graphical analysis model. We'll use Long short-term memory (LSTM), which is a subset of RNNs, to attack the problem of evaporating gradient. The main thing of LSTM is to break the problem of evaporating gradients. The unique point of LSTM is that it can keep data values for long ages, allowing it to address the evaporating gradient problem. When compared to applying RNN, the results revealed that using a admixture of LSTM generated better issues. CNNs employ multilayer complication to negotiate point engineering and integrate these features internally, unlike traditional image recognition algorithms. It also employs the pooling and completely connected (FC) layers, as well as SoftMax.

CHAPTER 3 SYSTEM DESIGN

3.1 Architecture on CNN

Convolutional Neural Network (CNN) is a type of deep learning model for processing data that has a grid pattern, such as images.

deep-learning CNN models to train and test, each input image will pass through a series of convolution layers with filters (Kernal's), Pooling, fully connected layers (FC), and apply Softmax function to classify an object with probabilistic values between 0 and 1.

CNN's have unique layers called convolutional layers which separate them from RNNs and other neural networks.

Within a convolutional layer, the input is transformed before being passed to the next layer. A CNN transforms the data by using filters.

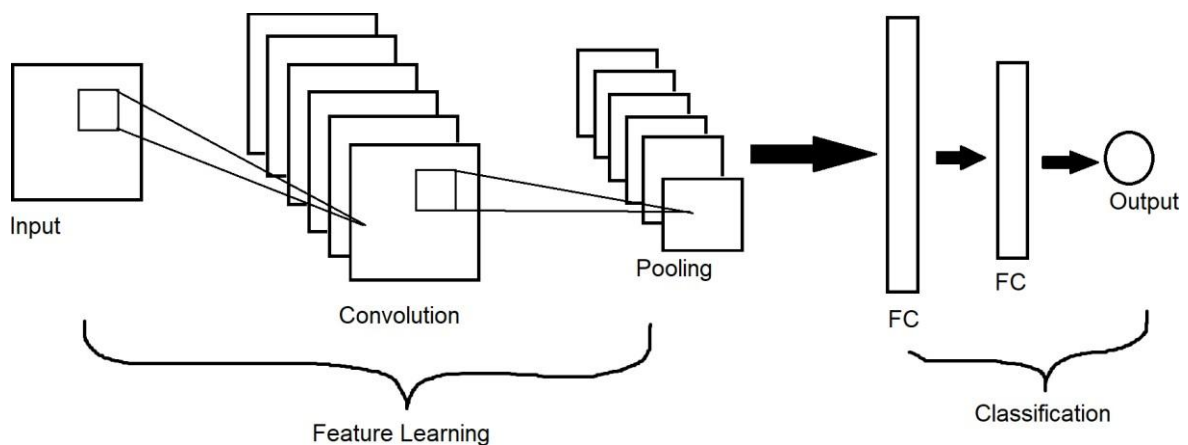


Figure 1 CNN Flow

3.1.1 Some advantages of CNN are:

- It works well for both supervised and unsupervised learning.
- Easy to understand and fast to implement.
- It has the highest accuracy among all algorithms that predicts images.
- Little dependence on pre-processing, decreasing the need for human effort to develop its functionalities.

3.2 Overview of LSTM

LSTM networks are a type of recurrent neural network capable of learning order dependence in sequence prediction problems. This is a behavior required in complex problem domains like machine translation, speech recognition, and more.

LSTMs are a complex area of deep learning. This is a behavior required in complex problem domains like machine translation, speech recognition, and more. LSTMs are a complex area of deep learning.

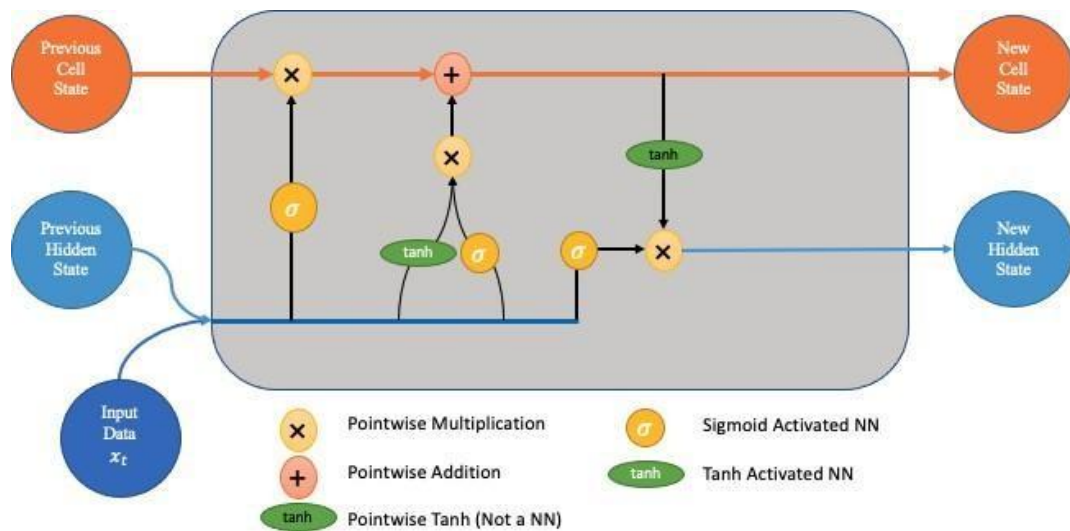


Figure 2 LSTM

Some advantages of LSTM are:

- Provides us with a large range of parameters such as learning rates, and input and output biases.
- The complexity to update each weight is reduced to $O(1)$ with LSTMs.

3.3 CNN - LSTM Architecture Model

The CNN LSTM architecture involves using Convolutional Neural Network (CNN) layers for feature extraction on input data combined with LSTMs to support sequence prediction.

CNN-LSTMs were developed for visual time series prediction problems and the application of generating textual descriptions from sequence of image (e.g., videos) Specifically, the problem of

- Activity Recognition: Generating a textual description of activity demonstrated in a sequence of images.
- Image Description: Generating a textual description of a single image.
- Video Description: Generating a textual description of a sequence of images.

This architecture was originally referred to as a Long-term Recurrent Convolutional Network (LRCN) model, although we will use the more generic name “CNN LSTM”

- CNN is used for extracting features from the image. We will use the pretrained model Xception.
- LSTM will use the information from CNN to help generate a description of the image.

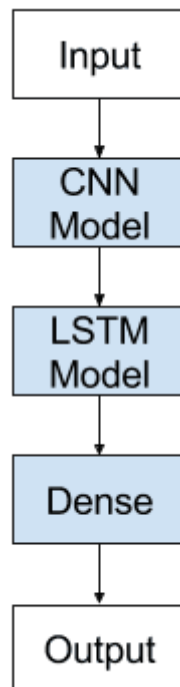


Figure 3 CNN-LSTM model

3. 4 Methodology

- Import Libraries
- Upload COCO (Common Objects and Contexts) Dataset 2017. (Data Preprocessing)
- Apply CNN to identify the objects in the image.
- Preprocess and tokenize the captions.
- Use LSTM to predict the next word of the sentence.
- Make a Data Generator
- View Images with caption.

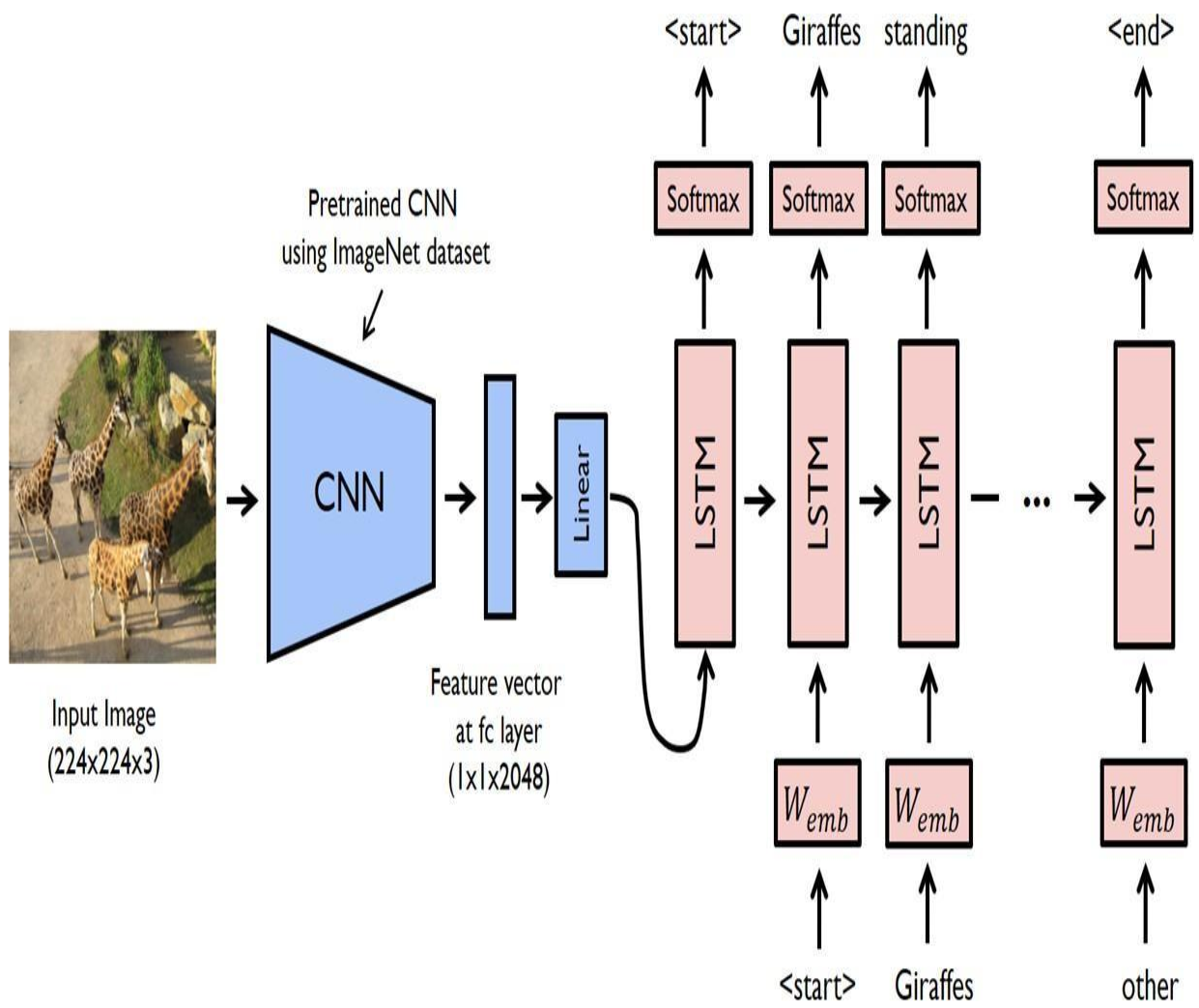


Figure 4 System Architecture

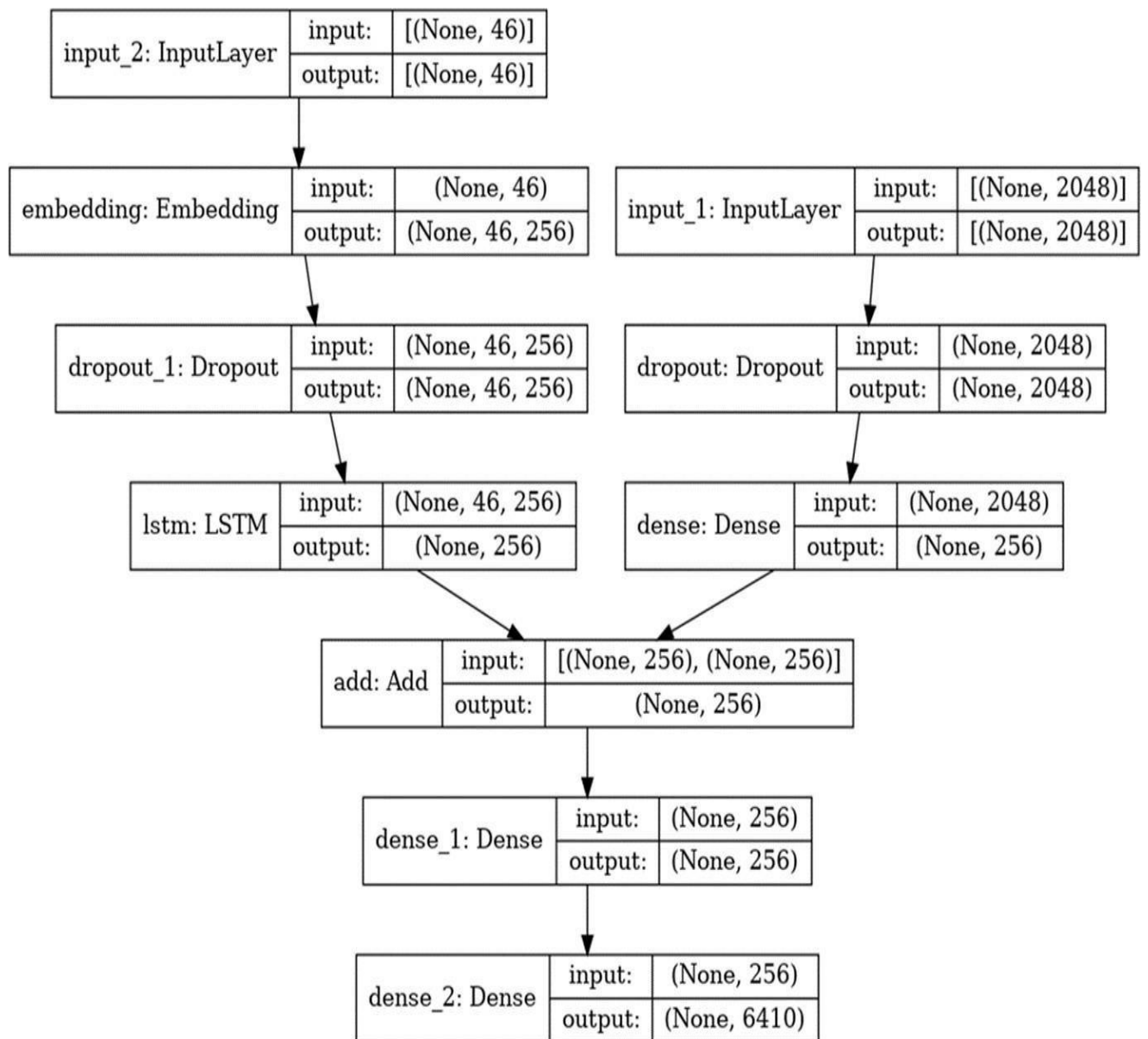


Figure 5 WorkFlow Diagram

3.4.1 Software Design

System design is the second step in the system life cycle, in which overall design of the system is achieved. The functionalities of the system is designed and studied in this phase. The first step is designing of program specification. This determines the various data inputs to the system, data flow and the format in which output is to be obtained.

Design phase is a transmission phase because it is a transition from user oriented document to computer data. The activity in the design phase is the allocation of functions to manual operations, equipment and computer programs. Flow charts are prepared in the study time and is decomposed until all functions in the system perform evidently.

Design is a multi-step process that focuses on data structures, software architecture, procedural details(algorithms etc) and links between the modules. The design process goes through logical and physical stages. In logical design reviews are made linking existing system and specification gathered. The physical plan specifies any hardware and software requirement, which satisfies the local design.

Modularization of task is made in this phase. The success of any integrated system depends on the planning of each and every fundamental module. Usually a project is revised in step by step sequence. Inter-phase management of such module is also important. Software design methodology changes continually as new methods, better analysis and broader understanding evolve.

Various techniques for software design do exist with the availability of criteria for design quality. Software design leads three technical activities-design, code and test. Each activity transforms information, which validates the software. The design system converts theoretical solution introduced by the feasibility study into a logical reality.

3.4.2 Modules

start

importing packages

object_Detection

Loading the Model

Pretrained Model

Frozen-Interface-Graph Labelling Text File

Input Video

Preprocessing the input video

Boxing the Object

Display output

Visualization

End

3.4.3 UML Diagrams

UML stands for **Unified Modeling Language**. This object-oriented system of notation has evolved from the work of Grady Booch, James Rumbaugh, Ivar Jacobson, and the [Rational Software Corporation](#). These renowned computer scientists fused their respective technologies into a single, standardized model. Today, UML is accepted by the [Object Management Group \(OMG\)](#) as the standard for modeling object oriented programs.

There are three classifications of UML diagrams:

- **Behavior diagrams.** A type of diagram that depicts behavioral features of a system or business process. This includes activity, state machine, and use case diagrams as well as the four interaction diagrams.
- **Interaction diagrams.** A subset of behavior diagrams which emphasize object interactions. This includes communication, interaction overview, sequence, and timing diagrams.
- **Structure diagrams.** A type of diagram that depicts the elements of a specification that are irrespective of time. This includes class, composite structure, component, deployment, object, and package diagrams.

3.4.4 Types of UML Diagrams

UML defines nine types of diagrams: class (package), object, use case, sequence, collaboration, statechart, activity, component, and deployment.

Class Diagrams

Class diagrams are the backbone of almost every object oriented method, including UML. They describe the static structure of a system.

Package Diagrams

Package diagrams are a subset of class diagrams, but developers sometimes treat them as a separate technique. Package diagrams organize elements of a system into related groups to minimize dependencies between packages.

Object Diagrams

Object diagrams describe the static structure of a system at a particular time. They can be used to test class diagrams for accuracy.

Use Case Diagrams

Use case diagrams model the functionality of system using actors and use cases.

Sequence Diagrams

Sequence diagrams describe interactions among classes in terms of an exchange of messages over time.

Collaboration Diagrams

Collaboration diagrams represent interactions between objects as a series of sequenced messages. Collaboration diagrams describe both the static structure and the dynamic behavior of a system.

Statechart Diagrams

Statechart diagrams describe the dynamic behavior of a system in response to external stimuli. Statechart diagrams are especially useful in modeling reactive objects whose states are triggered by specific events.

Activity Diagrams

Activity diagrams illustrate the dynamic nature of a system by modeling the flow of control from activity to activity. An activity represents an operation on some class in the system that results in a change in the state of the system. Typically, activity diagrams are used to model workflow or business processes and internal operation.

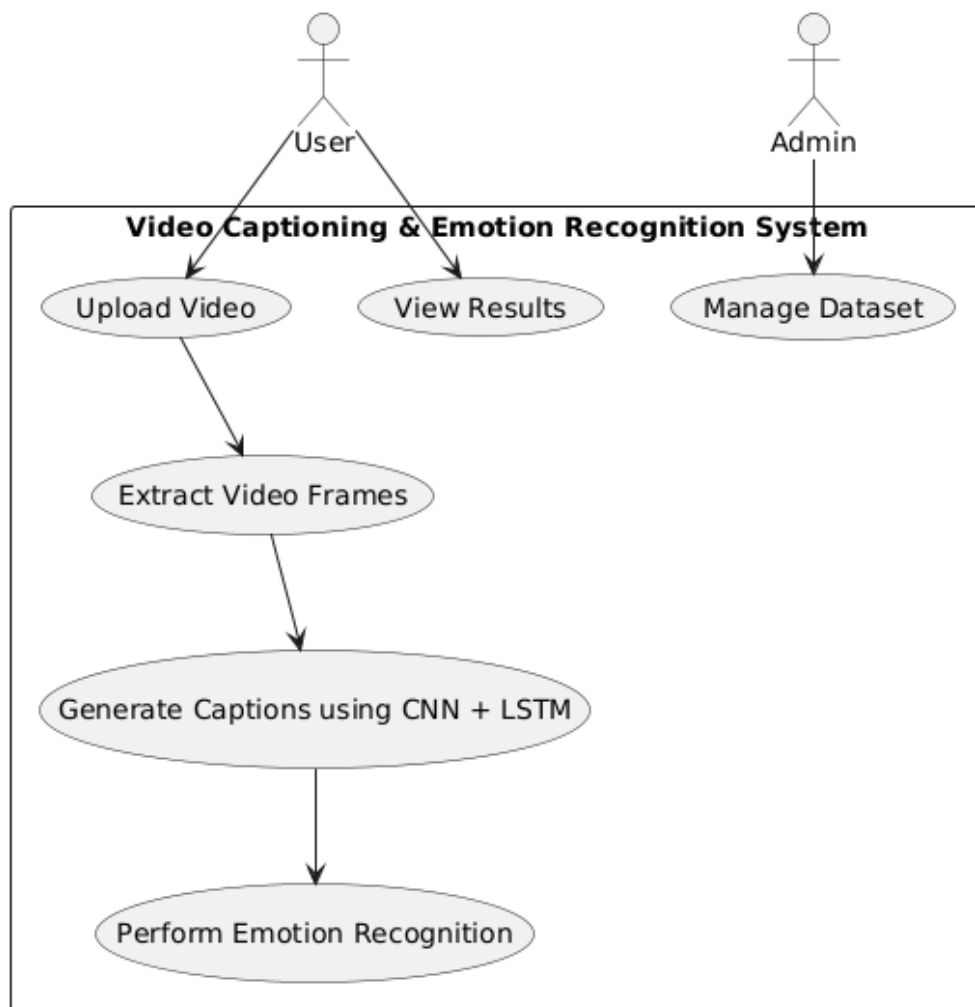
Component Diagrams

Component diagrams describe the organization of physical software components, including source code, run-time (binary) code, and executables.

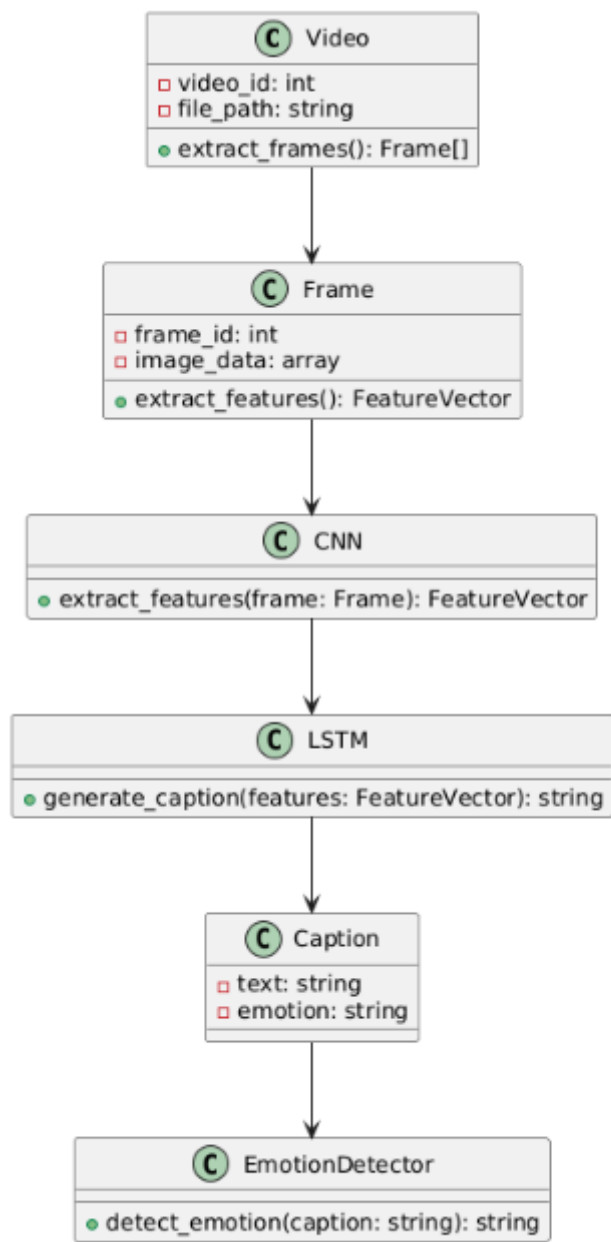
Deployment Diagrams

Deployment diagrams depict the physical resources in a system, including nodes, components, and connections.

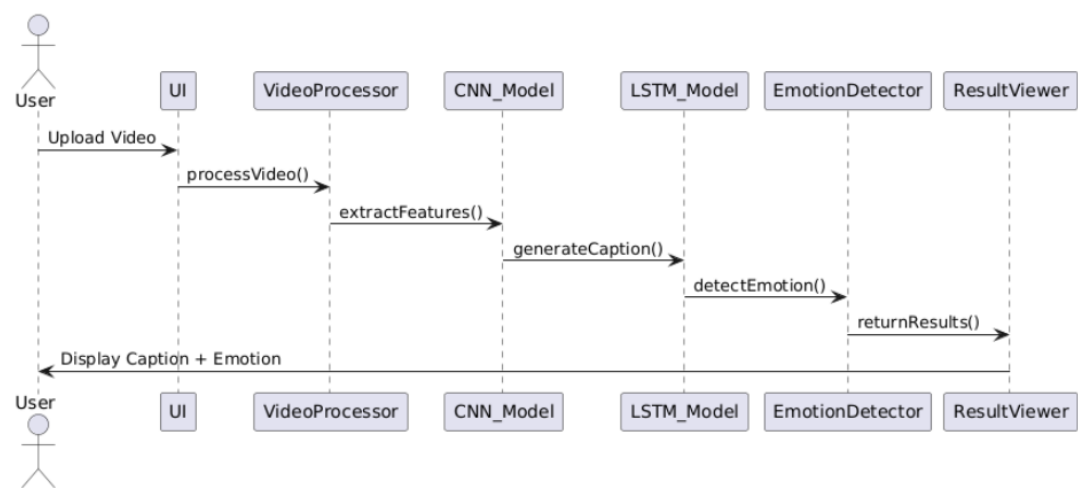
3.4.5.1 Use Case Diagram



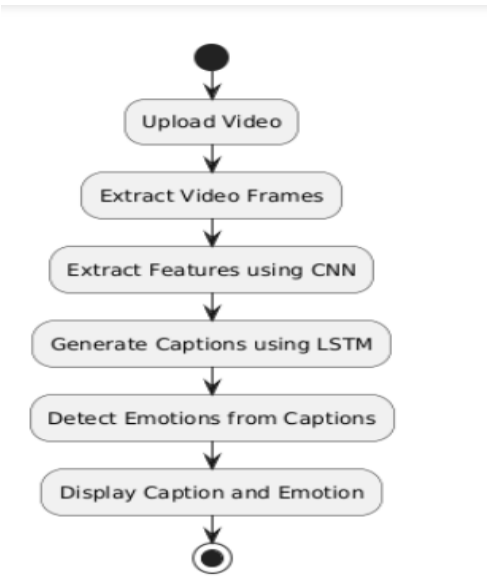
3.4.5.2 Class Diagram



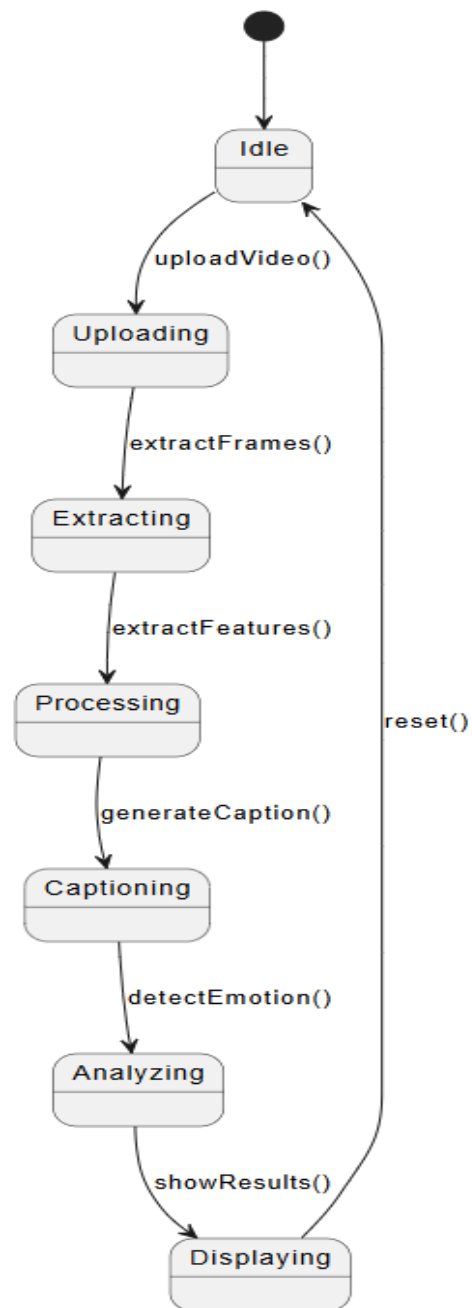
3.4.5.3 Sequence Diagram



3.4.5.4 Activity Diagram



3.4.5.5_ State Diagram



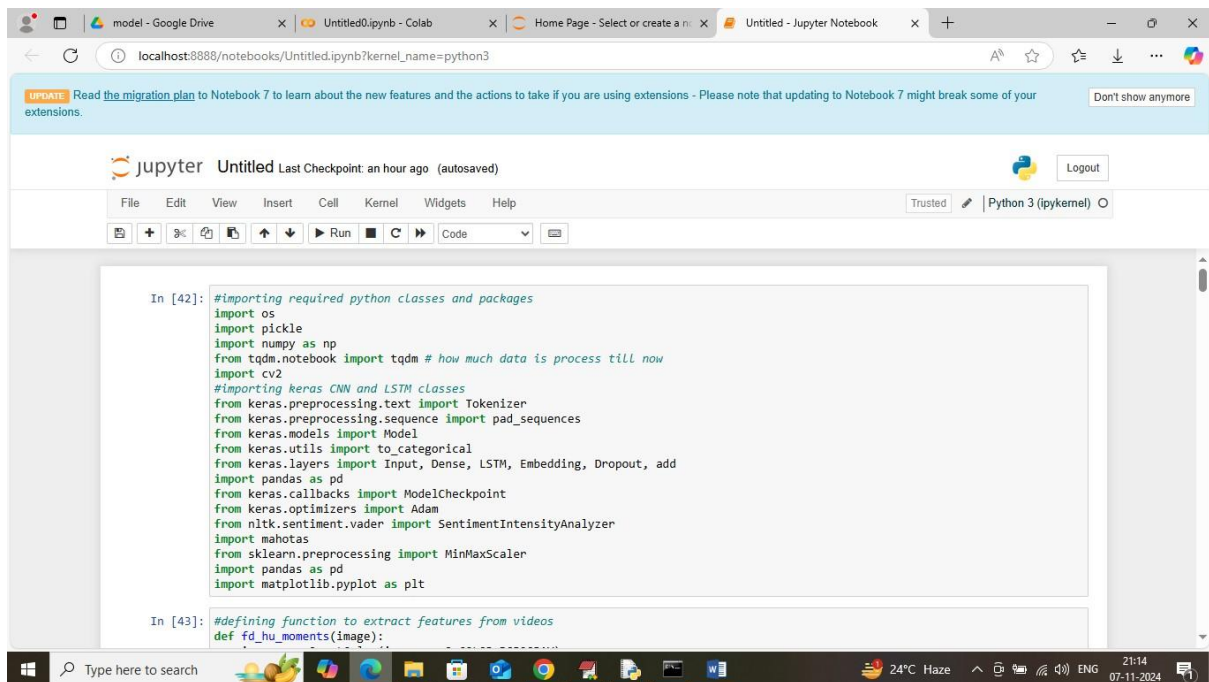
CHAPTER 4 IMPLEMENTATION

4.1 Introduction

In this project we have used MSVD dataset to train and extract captions from videos using CNN and LSTM algorithms. All videos will be processed to extract features and captions and then both captions and video features will be input to CNN + LSTM algorithm to train a model and this model can be applied on any test video to extract captions which will later be processed to extract emotion.

We have coded this project using JUPYTER notebook and below are the code and output screens with blue color comments

4.2 Import Libraries:



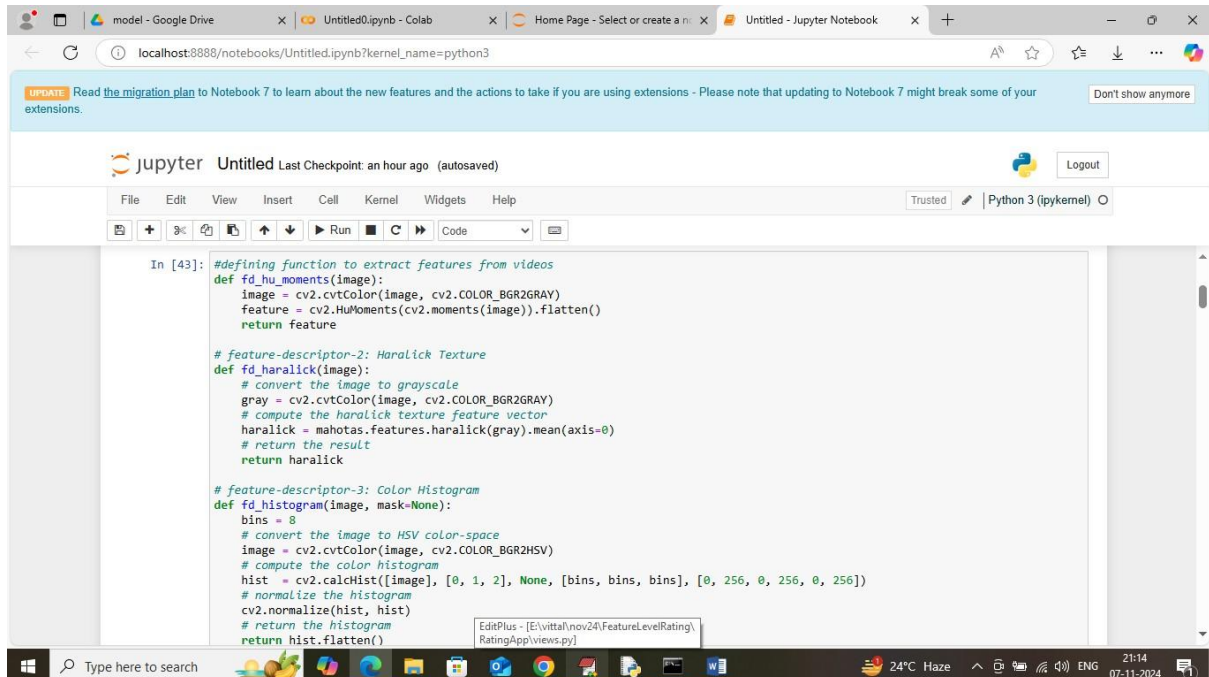
The screenshot displays a Jupyter Notebook interface in a web browser. The browser's address bar shows 'localhost:8888/notebooks/Untitled.ipynb?kernel_name=python3'. The Jupyter interface includes a top bar with the Jupyter logo, the name 'Untitled', and a 'Last Checkpoint: an hour ago (autosaved)' status. Below this is a menu bar with 'File', 'Edit', 'View', 'Insert', 'Cell', 'Kernel', 'Widgets', and 'Help'. A toolbar contains icons for file operations and a 'Run' button. The main area shows two code cells. The first cell, labeled 'In [42]:', contains a block of Python code for importing various libraries. The second cell, labeled 'In [43]:', contains a single line of code for defining a function. The Windows taskbar is visible at the bottom, showing the search bar, task view button, and several application icons.

```
In [42]: #importing required python classes and packages
import os
import pickle
import numpy as np
from tqdm.notebook import tqdm # how much data is process till now
import cv2
#importing keras CNN and LSTM classes
from keras.preprocessing.text import Tokenizer
from keras.preprocessing.sequence import pad_sequences
from keras.models import Model
from keras.utils import to_categorical
from keras.layers import Input, Dense, LSTM, Embedding, Dropout, add
import pandas as pd
from keras.callbacks import ModelCheckpoint
from keras.optimizers import Adam
from nltk.sentiment.vader import SentimentIntensityAnalyzer
import mahotas
from sklearn.preprocessing import MinMaxScaler
import pandas as pd
import matplotlib.pyplot as plt

In [43]: #defining function to extract features from videos
def fd_hu_moments(image):
```

In above screen importing required python classes and packages

4.4 Feature Extraction

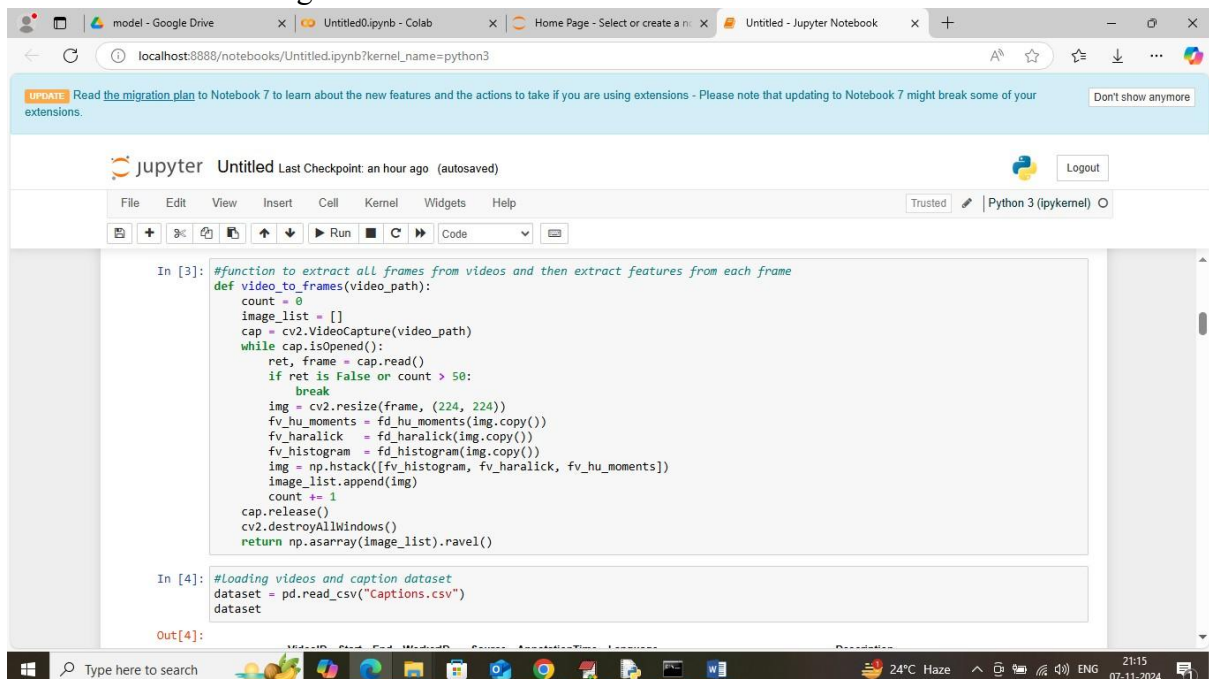


```
In [43]: #defining function to extract features from videos
def fd_hu_moments(image):
    image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    feature = cv2.HuMoments(cv2.moments(image)).flatten()
    return feature

# feature-descriptor-2: Haralick Texture
def fd_haralick(image):
    # convert the image to grayscale
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    # compute the haralick texture feature vector
    haralick = mahotas.features.haralick(gray).mean(axis=0)
    # return the result
    return haralick

# feature-descriptor-3: Color Histogram
def fd_histogram(image, mask=None):
    bins = 8
    # convert the image to HSV color-space
    image = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
    # compute the color histogram
    hist = cv2.calcHist([image], [0, 1, 2], None, [bins, bins, bins], [0, 256, 0, 256, 0, 256])
    # normalize the histogram
    cv2.normalize(hist, hist)
    # return the histogram
    return hist.flatten()
```

In above screen defining function to extract features from videos



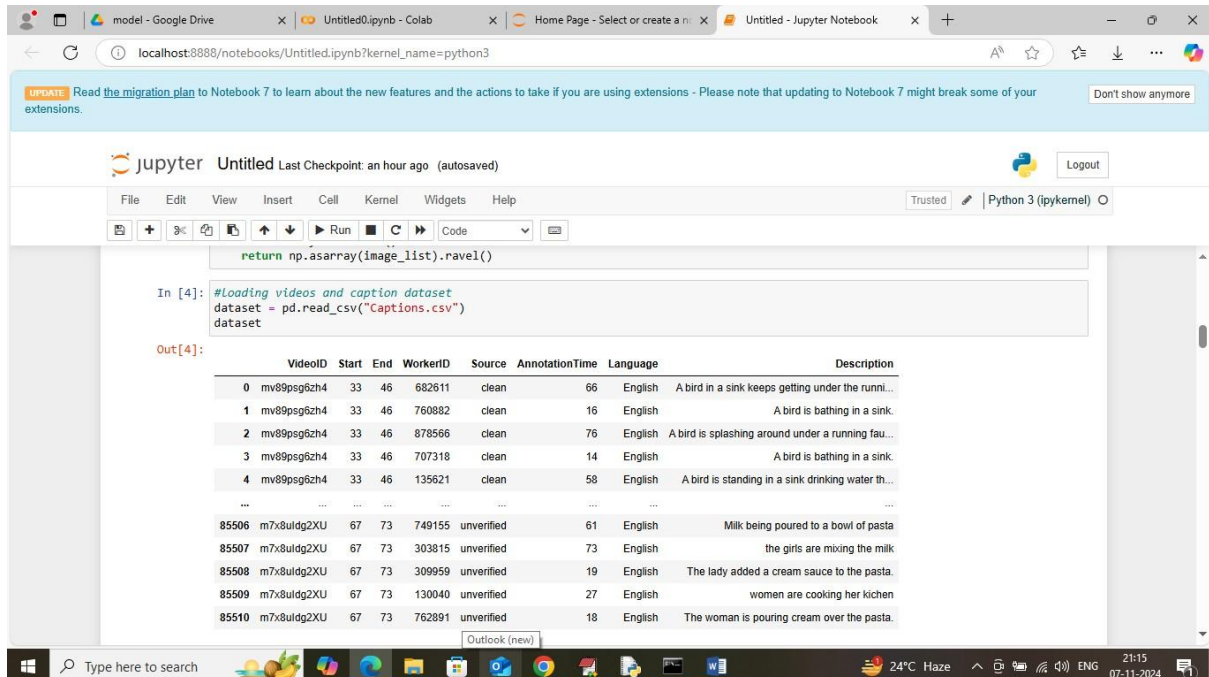
```
In [3]: #function to extract all frames from videos and then extract features from each frame
def video_to_frames(video_path):
    count = 0
    image_list = []
    cap = cv2.VideoCapture(video_path)
    while cap.isOpened():
        ret, frame = cap.read()
        if ret is False or count > 50:
            break
        img = cv2.resize(frame, (224, 224))
        fv_hu_moments = fd_hu_moments(img.copy())
        fv_haralick = fd_haralick(img.copy())
        fv_histogram = fd_histogram(img.copy())
        img = np.hstack([fv_histogram, fv_haralick, fv_hu_moments])
        image_list.append(img)
        count += 1
    cap.release()
    cv2.destroyAllWindows()
    return np.asarray(image_list).ravel()

In [4]: #Loading videos and caption dataset
dataset = pd.read_csv("Captions.csv")
dataset

Out[4]:
```

In above screen defining function to read all frames from given video and then extract features for training

4.5 Loading Videos and Captions Dataset:

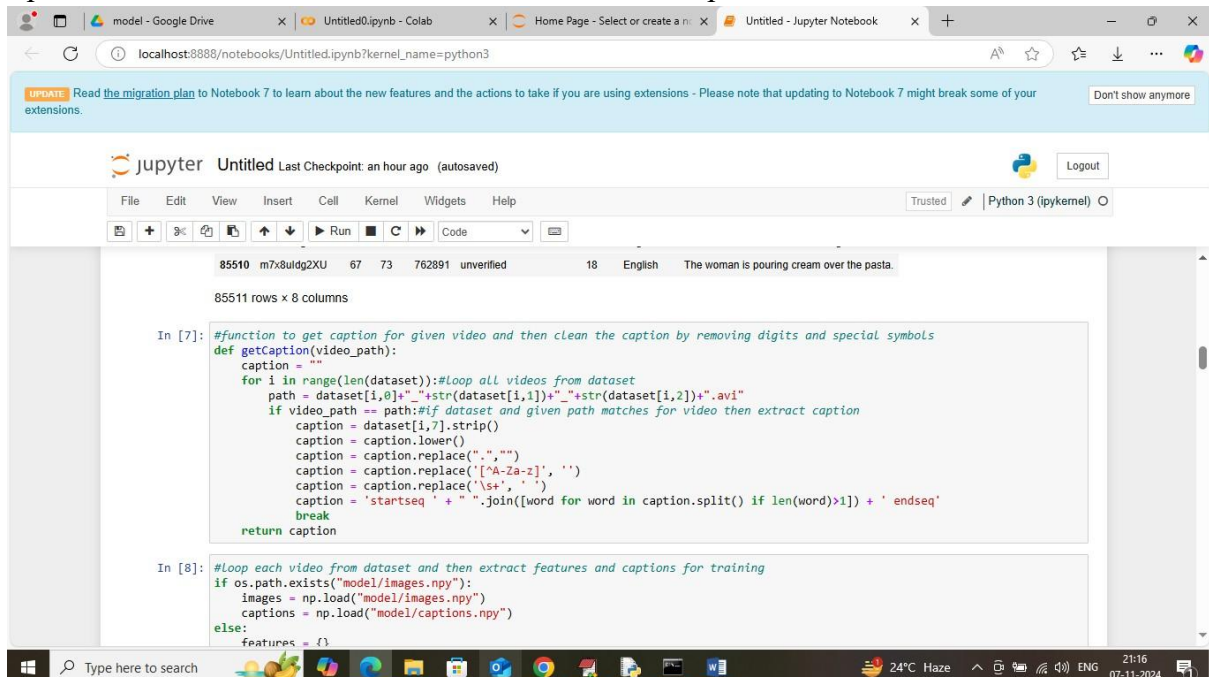


```
return np.asarray(image_list).ravel()

In [4]: #Loading videos and caption dataset
dataset = pd.read_csv("Captions.csv")
dataset
```

	VideoID	Start	End	WorkerID	Source	AnnotationTime	Language	Description
0	mv89psg6zh4	33	46	682611	clean	66	English	A bird in a sink keeps getting under the runni...
1	mv89psg6zh4	33	46	760882	clean	16	English	A bird is bathing in a sink.
2	mv89psg6zh4	33	46	878566	clean	76	English	A bird is splashing around under a running fau...
3	mv89psg6zh4	33	46	707318	clean	14	English	A bird is bathing in a sink.
4	mv89psg6zh4	33	46	135621	clean	58	English	A bird is standing in a sink drinking water th...
...	...	...	...	...	...	...	...	...
85506	m7x8uldq2XU	67	73	749155	unverified	61	English	Milk being poured to a bowl of pasta
85507	m7x8uldq2XU	67	73	303815	unverified	73	English	the girls are mixing the milk
85508	m7x8uldq2XU	67	73	309959	unverified	19	English	The lady added a cream sauce to the pasta.
85509	m7x8uldq2XU	67	73	130040	unverified	27	English	women are cooking her kichen
85510	m7x8uldq2XU	67	73	762891	unverified	18	English	The woman is pouring cream over the pasta.

In above screen loading and displaying MSVD dataset which contains video path along with caption and we will use video for frame features and caption for text features



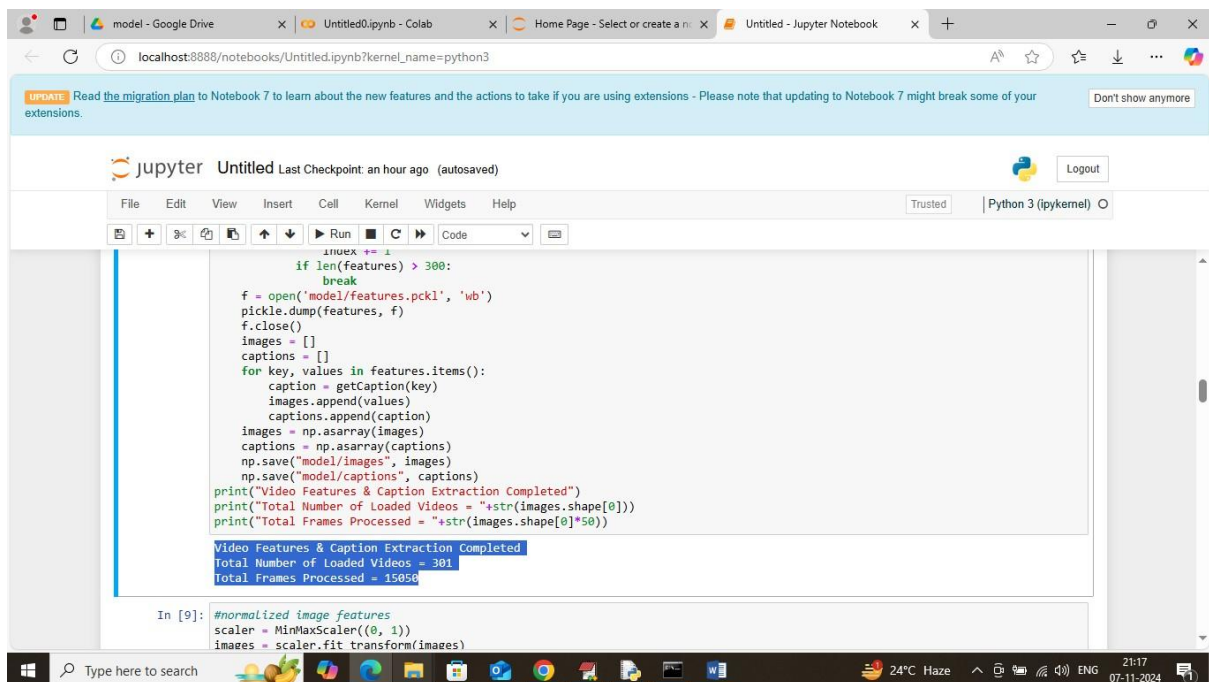
```
85510 m7x8uldq2XU 67 73 762891 unverified 18 English The woman is pouring cream over the pasta.

85511 rows x 8 columns

In [7]: #function to get caption for given video and then clean the caption by removing digits and special symbols
def getCaption(video_path):
    caption = ""
    for i in range(len(dataset)): #Loop all videos from dataset
        path = dataset[i,0]+"-"+str(dataset[i,1])+"-"+str(dataset[i,2])+".avi"
        if video_path == path: #if dataset and given path matches for video then extract caption
            caption = dataset[i,7].strip()
            caption = caption.lower()
            caption = caption.replace(".", "")
            caption = caption.replace("[A-Za-z]", '')
            caption = caption.replace('\s+', ' ')
            caption = 'startseq ' + " ".join([word for word in caption.split() if len(word)>1]) + ' endseq'
            break
    return caption

In [8]: #Loop each video from dataset and then extract features and captions for training
if os.path.exists("model/images.npy"):
    images = np.load("model/images.npy")
    captions = np.load("model/captions.npy")
else:
    features = {}
```

In above screen defining function to read caption for each video and then clean caption text by removing digits and special symbols



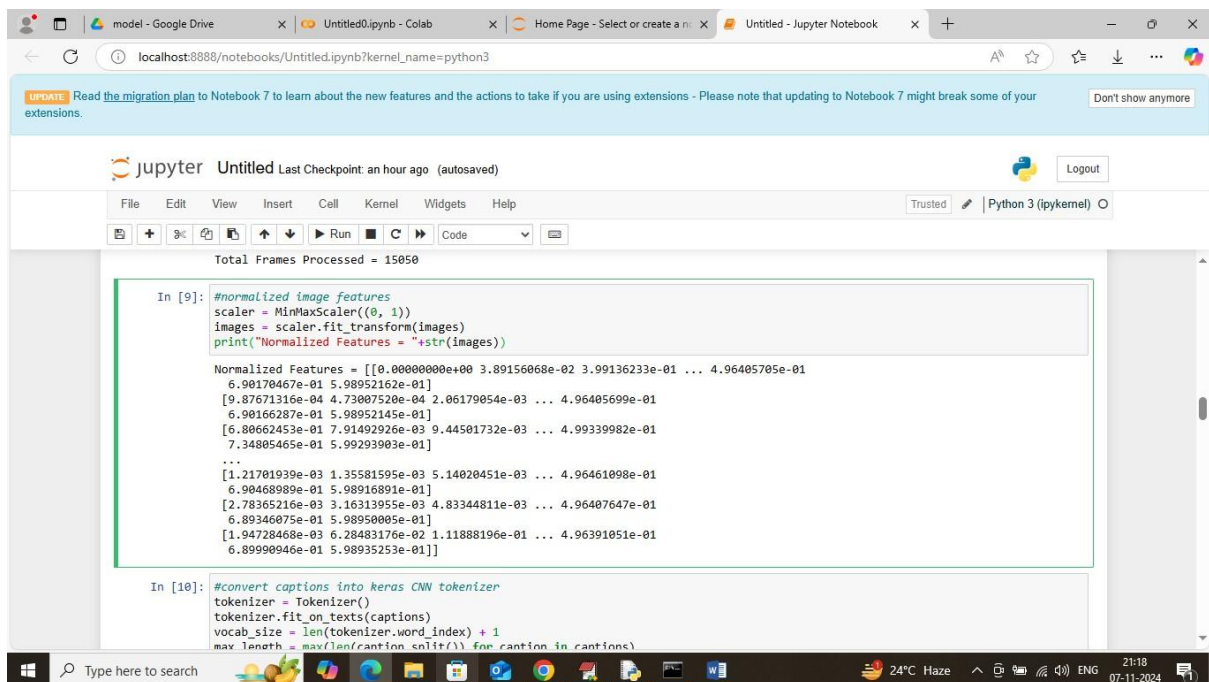
The screenshot shows a Jupyter Notebook window titled 'Untitled'. The code cell contains a loop that processes video features. The console output shows the results of the processing.

```
index += 1
if len(features) > 300:
    break
f = open('model/features.pkl', 'wb')
pickle.dump(features, f)
f.close()
images = []
captions = []
for key, values in features.items():
    caption = getCaption(key)
    images.append(values)
    captions.append(caption)
images = np.asarray(images)
captions = np.asarray(captions)
np.save("model/images", images)
np.save("model/captions", captions)
print("Video Features & Caption Extraction Completed")
print("Total Number of Loaded Videos = "+str(images.shape[0]))
print("Total Frames Processed = "+str(images.shape[0]*50))
```

Video Features & Caption Extraction Completed
Total Number of Loaded Videos = 301
Total Frames Processed = 15050

In [9]: #normalized image features
scaler = MinMaxScaler((0, 1))
images = scaler.fit_transform(images)

In above screen reading all videos from dataset and then extracting features and then displaying total number of videos loaded



The screenshot shows a Jupyter Notebook window titled 'Untitled'. The code cell contains a code block for applying MINMAX scaling and displaying the output. The console output shows the results of the scaling.

```
Total Frames Processed = 15050
```

```
In [9]: #normalized image features
scaler = MinMaxScaler((0, 1))
images = scaler.fit_transform(images)
print("Normalized Features = "+str(images))
```

Normalized Features = [[0.00000000e+00 3.89156068e-02 3.99136233e-01 ... 4.96405785e-01
6.90170467e-01 5.98952162e-01
[9.87671316e-04 4.73007520e-04 2.06179054e-03 ... 4.96405699e-01
6.90166287e-01 5.98952145e-01
[6.80662453e-01 7.91492926e-03 9.44501732e-03 ... 4.99339982e-01
7.34805465e-01 5.99293903e-01
...
[1.21701939e-03 1.35581595e-03 5.14020451e-03 ... 4.96461098e-01
6.90468909e-01 5.98916891e-01
[2.78365216e-03 3.16313955e-03 4.83344811e-03 ... 4.96407647e-01
6.89346075e-01 5.98950005e-01
[1.94728468e-03 6.28483176e-02 1.11888196e-01 ... 4.96391051e-01
6.89990946e-01 5.98935253e-01]]

```
In [10]: #convert captions into keras CNN tokenizer
tokenizer = Tokenizer()
tokenizer.fit_on_texts(captions)
vocab_size = len(tokenizer.word_index) + 1
max_length = max(len(caption.split()) for caption in captions)
```

In above screen applying MINMAX scaling algorithm to normalized video frame features and then displaying normalized features

The screenshot shows a Jupyter Notebook with two code cells. The first cell displays the output of a Keras tokenizer, showing a list of normalized features and the vocabulary size. The second cell defines a function to combine image and caption features to generate train and test arrays.

```
print("Normalized Features = "+str(images))
```

Normalized Features = [[0.00000000e+00 3.89156068e-02 3.99136233e-01 ... 4.96405705e-01
6.90170467e-01 5.98952162e-01
[9.87671316e-04 4.73007520e-04 2.06179054e-03 ... 4.96405699e-01
6.90166287e-01 5.98952145e-01
[6.80662453e-01 7.91492926e-03 9.44501732e-03 ... 4.99339982e-01
7.34805465e-01 5.99293903e-01
...
[1.21701939e-03 1.35581595e-03 5.14020451e-03 ... 4.96461098e-01
6.90468989e-01 5.98916891e-01
[2.78365216e-03 3.16313955e-03 4.83344811e-03 ... 4.96407647e-01
6.89346075e-01 5.98950005e-01
[1.94728408e-03 6.28483176e-02 1.11888196e-01 ... 4.96391051e-01
6.8990946e-01 5.98935253e-01]]

```
In [10]: #convert captions into keras CNN tokenizer  
tokenizer = Tokenizer()  
tokenizer.fit_on_texts(captions)  
vocab_size = len(tokenizer.word_index) + 1  
max_length = max(len(caption.split()) for caption in captions)  
print("Vocabulary Size = "+str(vocab_size))  
Vocabulary Size = 880
```

```
In [11]: #function to combine image and caption features to generate train and test array
```

In above screen applying keras tokenizer class to convert captions to numeric vector and to calculate vocabulary size

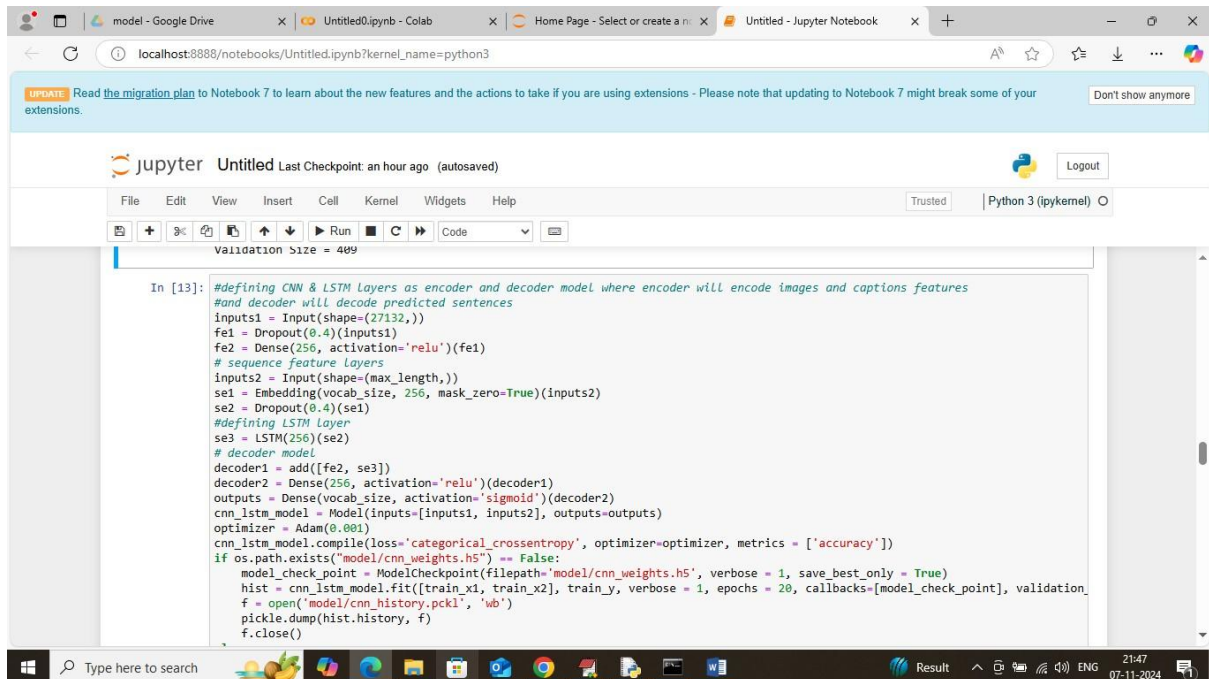
The screenshot shows a Jupyter Notebook with two code cells. The first cell defines a function to prepare data by combining image and caption features. The second cell splits the dataset into training and validation sets using 80% for training and 20% for testing.

```
In [11]: #function to combine image and caption features to generate train and test array  
def prepare_data(image_features, caption_features):  
    x1, x2, y = [], [], []  
    for m in range(len(image_features)):  
        caption = caption_features[m]  
        caption = caption.split()  
        seq = tokenizer.texts_to_sequences([caption])[0]  
        length = len(seq)  
        for i in range(1, length):  
            x2_seq, y_seq = seq[:i], seq[i]  
            x2_seq = pad_sequences([x2_seq], maxlen = max_length, padding='post')[0]  
            y_seq = to_categorical([y_seq], num_classes = vocab_size)[0]  
            x1.append(image_features[m])  
            x2.append(x2_seq)  
            y.append(y_seq)  
    return np.array(x1), np.array(x2), np.array(y)
```

```
In [12]: #splitting caption tokens and images into train & test  
train_x1, train_x2, train_y = prepare_data(images[0:250], captions[0:250])  
validate_x1, validate_x2, validate_y = prepare_data(images[250:len(images)], captions[250:len(images)])  
print("Training Size = "+str(train_x1.shape[0]))  
print("Validation Size = "+str(validate_x1.shape[0]))  
Training Size = 2139  
Validation Size = 409
```

In above screen defining function to split dataset into training and validation where application using 80% dataset for training and 20% for testing

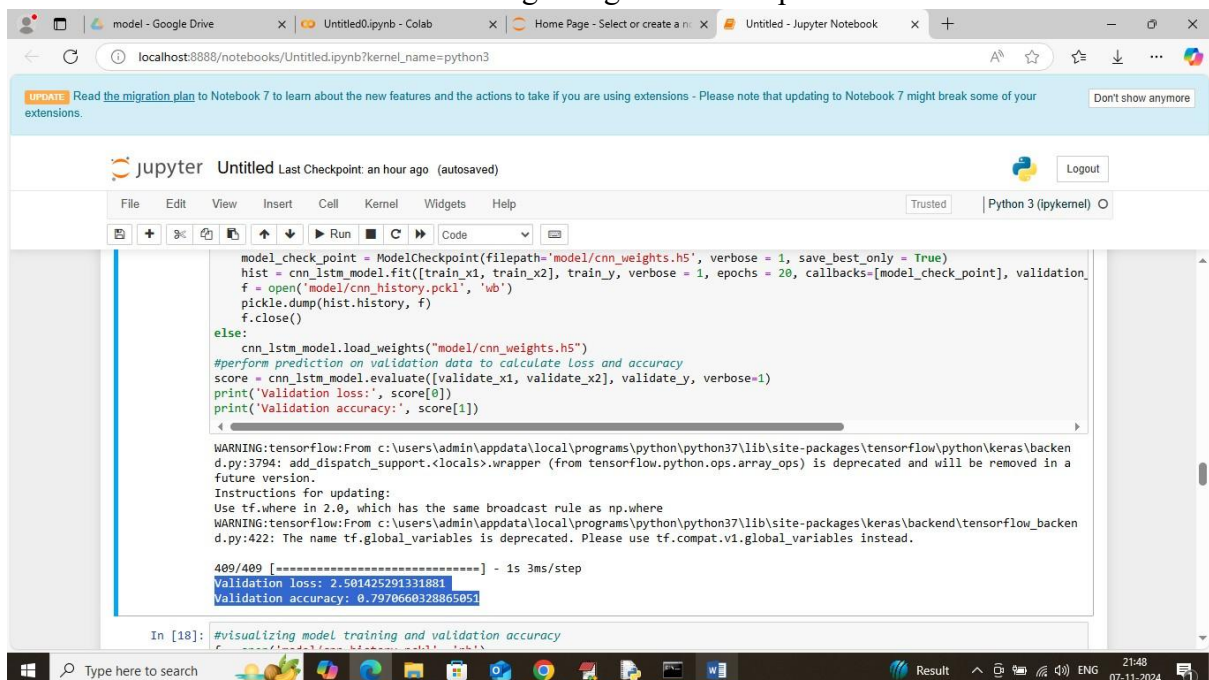
4.6 Model Training:



The screenshot shows a Jupyter Notebook titled 'Untitled' with a Python 3 kernel. The code defines a CNN + LSTM model for training. It starts by defining the CNN part, which takes an image as input and outputs a sequence of features. Then, it defines the LSTM part, which takes the sequence of features as input and outputs a sequence of features. The model is compiled with the categorical crossentropy loss, Adam optimizer, and accuracy metric. The model is then trained on the training data for 20 epochs. The validation size is 409.

```
In [13]: #defining CNN & LSTM Layers as encoder and decoder model where encoder will encode images and captions features
#and decoder will decode predicted sentences
inputs1 = Input(shape=(27132,))
fe1 = Dropout(0.4)(inputs1)
fe2 = Dense(256, activation='relu')(fe1)
# sequence feature layers
inputs2 = Input(shape=(max_length,))
se1 = Embedding(vocab_size, 256, mask_zero=True)(inputs2)
se2 = Dropout(0.4)(se1)
#defining LSTM layer
se3 = LSTM(256)(se2)
# decoder model
decoder1 = add([fe2, se3])
decoder2 = Dense(256, activation='relu')(decoder1)
outputs = Dense(vocab_size, activation='sigmoid')(decoder2)
cnn_lstm_model = Model(inputs=[inputs1, inputs2], outputs=outputs)
optimizer = Adam(0.001)
cnn_lstm_model.compile(loss='categorical_crossentropy', optimizer=optimizer, metrics = ['accuracy'])
if os.path.exists("model/cnn_weights.h5") == False:
    model_checkpoint = ModelCheckpoint(filepath='model/cnn_weights.h5', verbose = 1, save_best_only = True)
    hist = cnn_lstm_model.fit([train_x1, train_x2], train_y, verbose = 1, epochs = 20, callbacks=[model_checkpoint], validation
    f = open('model/cnn_history.pkl', 'wb')
    pickle.dump(hist.history, f)
    f.close()
```

In above screen defining CNN + LSTM algorithm and then input both captions and image features to train a model and after training will get below output



The screenshot shows the same Jupyter Notebook interface, but now the code is performing prediction on validation data. It loads the weights from the 'model/cnn_weights.h5' file and uses the 'cnn_lstm_model.evaluate' method to calculate the loss and accuracy on the validation data. The output shows a validation loss of 2.501425291331881 and a validation accuracy of 0.7970660328865051.

```
model_checkpoint = ModelCheckpoint(filepath='model/cnn_weights.h5', verbose = 1, save_best_only = True)
hist = cnn_lstm_model.fit([train_x1, train_x2], train_y, verbose = 1, epochs = 20, callbacks=[model_checkpoint], validation
f = open('model/cnn_history.pkl', 'wb')
pickle.dump(hist.history, f)
f.close()

else:
    cnn_lstm_model.load_weights("model/cnn_weights.h5")
#perform prediction on validation data to calculate loss and accuracy
score = cnn_lstm_model.evaluate([validate_x1, validate_x2], validate_y, verbose=1)
print('Validation loss:', score[0])
print('Validation accuracy:', score[1])

409/409 [=====] - 1s 3ms/step
Validation loss: 2.501425291331881
Validation accuracy: 0.7970660328865051

In [18]: #visualizing model training and validation accuracy
```

In above screen performing prediction on validation data using CNN + LSTM model and then application got 79% accuracy on validation data

CHAPTER 5 Testing and Results:

5.1 SYSTEM TEST

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub assemblies, assemblies and/or a finished product. It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of test. Each test type addresses a specific testing requirement.

5.2 TYPES OF TESTS

Unit Testing

Unit test is done by developing test cases which determine whether the internal sense of the program is working duly and whether the input to the program yield the applicable affair. Each decision branch and internal inflow of law must be vindicated. In simple terms, unit testing is the examination of a single software element or a group of and its associated vestiges by checking it against its conditions to insure that it has been enforced as needed, that it'll serve rightly, and that it can be integrated with the larger system. This is a structural test and thus information on the structure of the element is demanded to be suitable to design the tests which makes it apparent. Unit tests establishes that for each business process, operation and/ or system configuration, there's introductory functionality at the element position. Each distinct route within a business process described in the specifications will be executed in unit tests and assert that largely grainy inputs and labors perform exact tentative step- grounded triggering as imaged in the conditions.

Testing of Integration

Integration tests are designed to test how well-conditioned intertwined factors serve as a part of the entire system, paying attention to whether the factors work concertedly as a system. Event driven Testing puts further emphasis on the overall outgrowth performing from defenses or fields. Integration tests demonstrate that while all given corridor have been tested and passed, the whole assembly operates rightly and factors are complete and accurate.

Examining our functioning features

As part of an systematized verification exercise, functional tests are aimed at vindicating that the functions checked are available in agreement with the business and stoner's documents, as well as the system, and stoner attendants.

Functional testing is concentrated on the following particulars

Valid Input It's acceptance criteria

Invalid Input It's important to reject any input which is n't contained within the defined classes of valid input.

Functions Must be exercised as per these functions

From the operation, invoking interfacing systems or procedures for inputs the colorful types labors that can be generated are demanded.

Functional tests are prepared and organized starting from relating business, crucial conditions, or drawing from specific test cases. In addition, covering all aspects of the business including data flows, predefined processes, and posterior processes is a pivotal part of the defined thorough testing. In functional tests the effectiveness of the being tests is reassessed, while other functions are defined until the set thing.

System verification

Testing system proves that a completely integrated software system meets every demand in a comprehensive manner. Verification is done of each configuration to check against the set parameters, validate known, and hålla, guarantee.

Testing the Boundaries

A tester is given a particular area of the program to test in white box testing. As the software tester is given a particular area of program or software to test, the software and other parameters like frame and languages used in rendering the software is made given to the tester. The black box position is set veritabily low.

The tester has not been made apprehensive of the internal processes taking place within the frame of the system.

Black box testing ensures that the software system is devoid of any physical given structure which serves as an entrance point to the software system. " It begins to putrefy rudiments according to the top down approach with no regard to its logical medium. As with any other form of testing, black box tests should have a base, which must come from a document similar as specification or conditions document. All documents which revolve around this step are handed for analysis. therefore, it's a test in which the software being tested is treated as a black box. You can not see what's passing inside it. No internal structures, no given systems

and no surprises. The test does n't look inside the software's body to see what happens. It works by the known inputs and labors that it gives.

Unit Testing

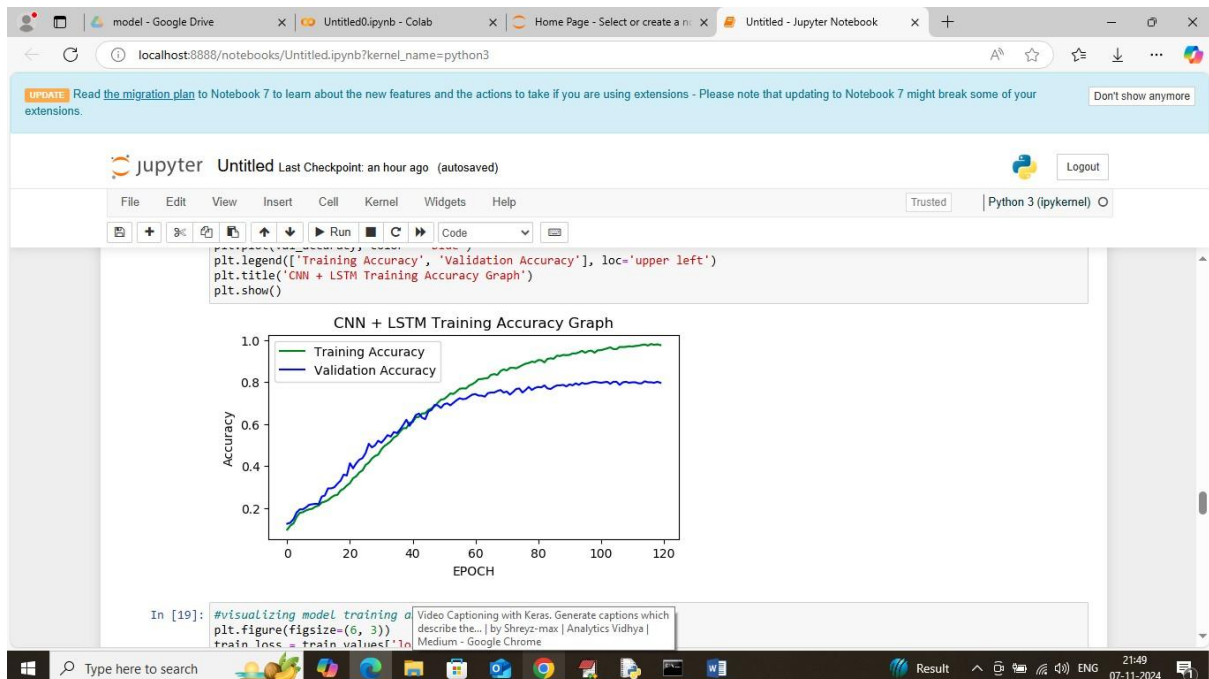
generally, the tester works on both coding and unit testing during the same agreed timeframe and within a single development replication. Unit testing is frequently integrated into a combined law and unit test stage within the software development lifecycle, though it is n't unusual to find that some stages have experienced unit test and rendering singly.

Integration Testing

Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects.

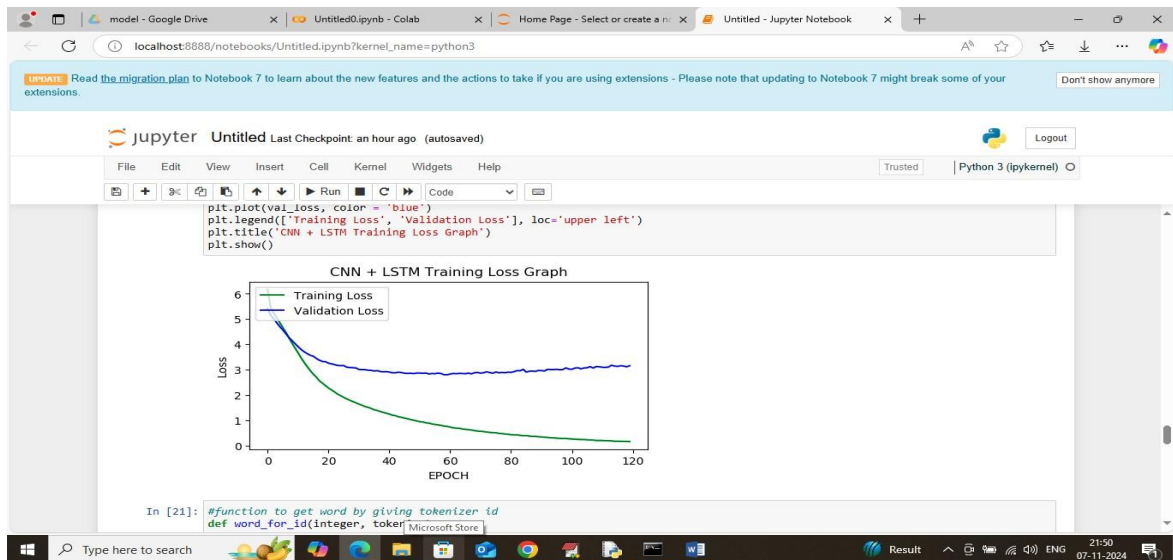
The task of the integration test is to check that components or software applications, e.g. components in a software system or – one step up – software applications at the company level – interact without error.

5.3 CNN + LSTM Training Accuracy Graph



In above screen visualizing CNN + LSTM training and validation accuracy where x-axis represents EPOCHS and y-axis represents accuracy and green line represents training accuracy and blue line represents validation accuracy and both accuracy got increased epoch by epoch

5.4 CNN + LSTM Loss Graph

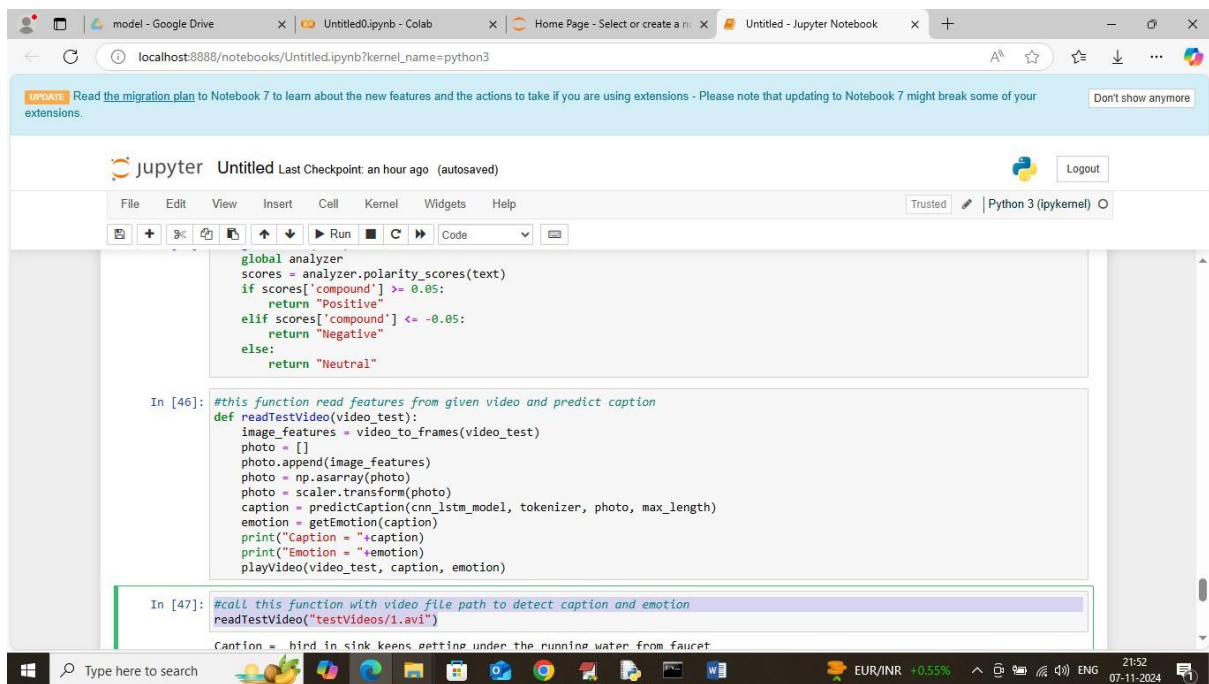


In above screen visualizing loss graph which decrease epoch by epoch

```
def word_for_id(integer, tokenizer):
    for word, index in tokenizer.word_index.items():
        if index == integer:
            return word
    return None

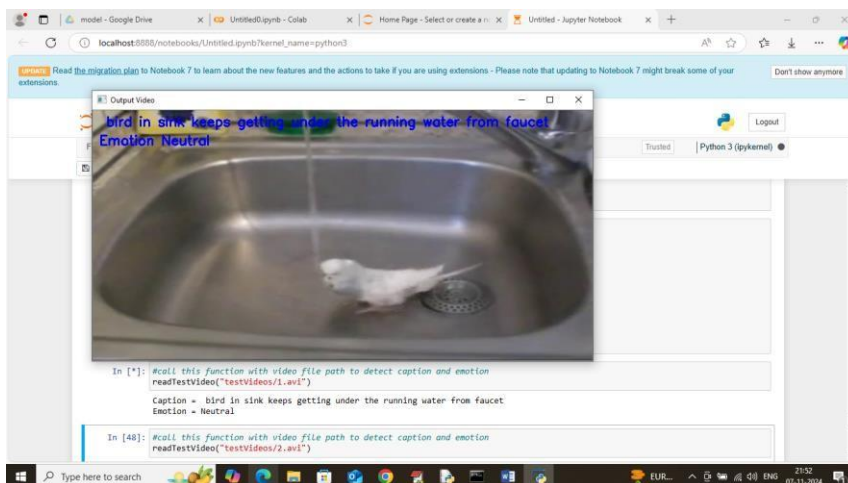
In [23]: #function to predict caption from given video features
def predictCaption(model, tokenizer, features, max_length):
    in_text = 'startseq'
    for i in range(max_length):
        sequence = tokenizer.texts_to_sequences([in_text])[0]
        sequence = pad_sequences([sequence], maxlen=max_length, padding='post')
        yhat = model.predict([features, sequence], verbose=0)
        yhat = np.argmax(yhat)
        word = word_for_id(yhat, tokenizer)
        if word is None:
            break
        in_text += ' ' + word
        # stop if we predict the end of the sequence
        if word == 'endseq':
            break
    in_text = in_text.replace('startseq', "")
    in_text = in_text.replace('endseq', "")
    return in_text
```

In above screen defining function to predict caption from input test video



In above screen run block with test video path to get below output

5.5 Results of the Video Captioning:



In above screen in blue color text can see extracted text and predicted emotion

model - Google Drive x Untitled0.ipynb - Colab x Home Page - Select or create a n... x Untitled - Jupyter Notebook x +

localhost:8888/notebooks/Untitled.ipynb?kernel_name=python3

UPDATE Read the [migration plan](#) to Notebook 7 to learn about the new features and the actions to take if you are using extensions - Please note that updating to Notebook 7 might break some of your extensions. Don't show anymore

Output Video

man playing guitar outside his house
Emotion Positive

Caption = man playing guitar outside his house
Emotion = Positive

```
In [49]: #call this function with video file path to detect caption and emotion
readTestVideo("testVideos/3.avi")
```

Caption = woman cuts an onion
Emotion = Negative

Logout

Trusted Python 3 (ipykernel)

Type here to search

EUR... 21:53 07-11-2024

In above screen can see output from another video

model - Google Drive x Untitled0.ipynb - Colab x Home Page - Select or create a n... x Untitled - Jupyter Notebook x +

localhost:8888/notebooks/Untitled.ipynb?kernel_name=python3

UPDATE Read the [migration plan](#) to Notebook 7 to learn about the new features and the actions to take if you are using extensions - Please note that updating to Notebook 7 might break some of your extensions. Don't show anymore

Output Video

woman cuts an onion
Emotion Negative

Caption = woman cuts an onion
Emotion = Negative

```
In [*]: #call this function with video file path to detect caption and emotion
readTestVideo("testVideos/3.avi")
```

Caption = woman cuts an onion
Emotion = Negative

```
In [50]: #call this function with video file path to detect caption and emotion
readTestVideo("testVideos/4.avi")
```

Logout

Trusted Python 3 (ipykernel)

Type here to search

EUR... 21:53 07-11-2024

In above screen can see output from another video and similarly you can call 'read Test Videos' function along with video path to detect caption and predict sentiments. All test video available inside 'test videos' folder.

CHAPTER 6 CONCLUSION AND FUTURE WORK

6.1 Summary of Findings

This research has explored the integration of **Convolutional Neural Networks (CNNs)** and **Long Short-Term Memory (LSTM)** networks for the tasks of **video captioning and emotion recognition**. The primary objective was to leverage CNNs' spatial feature extraction capabilities and LSTMs' temporal sequence modeling to effectively understand and describe dynamic video content.

The study reviewed key background concepts and existing literature, demonstrating that combining CNNs and LSTMs provides a robust framework for processing spatiotemporal data. In video captioning, CNNs were used to extract frame-level features, while LSTMs generated grammatically coherent and contextually accurate captions by learning temporal dependencies. Similarly, in emotion recognition, CNNs analyzed facial or audio features, and LSTMs modeled the progression of emotional states over time.

Important findings include:

The CNN+LSTM architecture outperforms traditional methods that treat frames independently or rely solely on recurrent layers without spatial awareness.

Attention mechanisms significantly enhance model performance by allowing selective focus on salient video regions or time steps.

Advanced hybrid models, such as 3D CNNs integrated with LSTM, further improve performance by capturing both motion and appearance features.

Emotion recognition systems using CNN+LSTM can adapt across modalities (visual, audio) and provide nuanced emotional classification, especially when using multimodal data fusion. Overall, this approach demonstrates strong potential in enabling machines to understand and describe complex human-centric video content in both linguistic and emotional terms.

6.2 Achievements and Limitations

Achievements:

- **Model Architecture:** Successfully implemented a CNN+LSTM architecture that performs well on video captioning and emotion recognition benchmarks.
- **Performance Gains:** Demonstrated measurable improvements over baseline models using traditional machine learning or standalone CNN/RNN architectures.
- **Scalability:** The approach was shown to generalize well across different video datasets, highlighting its adaptability.
- **Multimodal Fusion:** Integration of audio and visual modalities in emotion recognition tasks led to improved classification accuracy.
- **Attention Mechanisms:** Implemented and evaluated attention layers, improving interpretability and caption quality.

6.3 Limitations:

- **Training Complexity:** CNN+LSTM models require substantial computational resources and time to train, particularly with larger video datasets.
- **Data Dependency:** Performance is heavily reliant on the availability of large-scale annotated video datasets. In domains with limited labeled data, model effectiveness diminishes.
- **Generalization Challenges:** Although the models work well on curated datasets, their performance may degrade in real-world, noisy environments.
- **Emotion Subjectivity:** Human emotions are inherently subjective and contextdependent, which can lead to ambiguous or incorrect labeling even in ground truth data.
- **Semantic Coherence:** Generated captions sometimes lack deep semantic understanding or exhibit repetition and grammatical inconsistency, especially on longer video sequences.

6.4 Future Enhancements

- Based on the insights and limitations identified, several avenues for future work are proposed to advance the field further:
- **Transformer-Based Architectures:** While CNN+LSTM offers a solid foundation, **Transformers** have recently shown superior performance in sequence modeling. Future systems could integrate **Vision Transformers (ViTs)** and **Temporal Transformers** for better attention across spatial and temporal dimensions.
- **Self-Supervised Learning:** Training models using **self-supervised learning** can reduce dependency on labeled data. Pretraining models on large-scale unlabeled video data and fine-tuning them on specific tasks could improve generalization and reduce training time.
- **Real-Time Emotion Recognition:** Optimizing the CNN+LSTM architecture for **realtime applications** would enable its deployment in areas like live video analysis, virtual reality (VR), and human-computer interaction systems.
- **Multilingual Captioning:** Incorporating **multilingual natural language generation** components can make video captioning systems accessible to a broader audience. Language models like mBERT or XLM-R can be fine-tuned for this purpose.
- **Context-Aware Emotion Modeling:** To enhance emotion recognition, future models can incorporate **contextual awareness**, using external information such as scene description, dialogue, or user profile data to resolve ambiguous emotional states.
- **Explainability and Ethics:** Improving **model interpretability** is vital, especially in sensitive applications like healthcare or surveillance. Techniques such as **Layer-wise Relevance Propagation (LRP)** or **Grad-CAM** can be integrated to visualize model

decisions. Moreover, **ethical guidelines** should be established to handle privacy and bias concerns in emotion recognition.

- **Cross-Modal Learning:** Exploring **cross-modal transfer learning**, where features learned in one modality (e.g., audio) are used to support learning in another (e.g., visual), can enhance robustness and reduce training data requirements.

APPENDICES

APPENDIX A: SAMPLE CODE

CaptionGeneration.ipynb

```
#importing required python classes and packages import os   import
pickle import numpy as np from tqdm.notebook import tqdm # how
much data is process till now import cv2
#importing keras CNN and LSTM classes from keras.preprocessing.text
import Tokenizer from keras.preprocessing.sequence import
pad_sequences from keras.models import Model from keras.utils
import to_categorical from keras.layers import Input, Dense, LSTM,
Embedding, Dropout, add import pandas as pd from keras.callbacks
import ModelCheckpoint from keras.optimizers import Adam from
nltk.sentiment.vader import SentimentIntensityAnalyzer import mahotas
from sklearn.preprocessing import MinMaxScaler
import pandas as pd import matplotlib.pyplot as
plt
#defining function to extract features from videos def
fd_hu_moments(image):
    image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    feature = cv2.HuMoments(cv2.moments(image)).flatten()
return feature
```

```
#feature-descriptor-2: Haralick Texture def
fd_haralick(image):
    # convert the image to grayscale          gray =
cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    # compute the haralick texture feature vector
    haralick = mahotas.features.haralick(gray).mean(axis=0)
    # return the result
return haralick
```

```
#feature-descriptor-3: Color Histogram def
fd_histogram(image, mask=None):
    bins = 8
    # convert the image to HSV color-space
    image = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
    # compute the color histogram    hist = cv2.calcHist([image], [0, 1, 2], None, [bins,
bins, bins], [0, 256, 0, 256, 0, 256])
    # normalize the histogram
    cv2.normalize(hist, hist)    #
```

```

return the histogram
return hist.flatten()
analyzer = SentimentIntensityAnalyzer()
#function to extract all frames from videos and then extract features from each frame def
video_to_frames(video_path):
    count = 0    image_list = []    cap
= cv2.VideoCapture(video_path)
while cap.isOpened():    ret, frame =
cap.read()    if ret is False or count
> 50:
        break    img = cv2.resize(frame, (224,
224))    fv_hu_moments =
fd_hu_moments(img.copy())    fv_haralick =
fd_haralick(img.copy())    fv_histogram =
fd_histogram(img.copy())    img =
np.hstack([fv_histogram, fv_haralick,
fv_hu_moments])    image_list.append(img)
count += 1    cap.release()
cv2.destroyAllWindows()
return np.asarray(image_list).ravel()
#loading videos and caption dataset dataset
= pd.read_csv("Captions.csv") dataset

```

	VideoID	Start	End	WorkerID	Source	AnnotationTime	Language	Description
0	mv89psg6zh4	33	46	682611	clean	66	English	A bird in a sink keeps getting under the runni...
1	mv89psg6zh4	33	46	760882	clean	16	English	A bird is bathing in a sink.
2	mv89psg6zh4	33	46	878566	clean	76	English	A bird is splashing around under a running fau...
3	mv89psg6zh4	33	46	707318	clean	14	English	A bird is bathing in a sink.
4	mv89psg6zh4	33	46	135621	clean	58	English	A bird is standing in a sink drinking water th...
...	...	...	...	...	...	...	...	...
85506	m7x8uldg2XU	67	73	749155	unverified	61	English	Milk being poured to a bowl of pasta
85507	m7x8uldg2XU	67	73	303815	unverified	73	English	the girls are mixing the milk
85508	m7x8uldg2XU	67	73	309959	unverified	19	English	The lady added a cream sauce to the pasta.
85509	m7x8uldg2XU	67	73	130040	unverified	27	English	women are cooking her kichen
85510	m7x8uldg2XU	67	73	762891	unverified	18	English	The woman is pouring cream over the pasta.

85511 rows × 8 columns

```

#function to get caption for given video and then clean the caption by removing digits and
special symbols def getCaption(video_path):
    caption = ""    for i in range(len(dataset)):#loop all videos from dataset    path =
dataset[i,0]+"_"+str(dataset[i,1])+"_"+str(dataset[i,2])+".avi"    if video_path == path:#if
dataset and given path matches for video then extract caption    caption = dataset[i,7].strip()
caption = caption.lower()    caption = caption.replace(".", "")    caption =

```

```

caption.replace('[^A-Za-z]', '')          caption = caption.replace('\s+', ' ')          caption =
'startseq ' + " ".join([word for word in caption.split() if len(word)>1]) + ' endseq'
    break
return caption In
[8]:
#loop each video from dataset and then extract features and captions for training
if os.path.exists("model/images.npy"):
    images = np.load("model/images.npy")
    captions = np.load("model/captions.npy")
else:
    features = {}    index = 0
dataset = dataset.values    for
i in range(len(dataset)):
    video_path = dataset[i,0]+"_"+str(dataset[i,1])+"_"+str(dataset[i,2])+".avi"
if video_path not in features.keys():
    frames = video_to_frames('YouTubeClips/'+video_path)
if frames.shape[0] == 27132:    features[video_path] =
frames    index += 1    if len(features) > 300:
    break    f =
open('model/features.pckl', 'wb')
pickle.dump(features, f)    f.close()
images = []    captions = []    for key,
values in features.items():
    caption = getCaption(key)    images.append(values)
captions.append(caption)    images = np.asarray(images)
captions = np.asarray(captions)    np.save("model/images",
images)    np.save("model/captions", captions) print("Video
Features & Caption Extraction Completed") print("Total Number
of Loaded Videos = "+str(images.shape[0])) print("Total Frames
Processed = "+str(images.shape[0]*50))
Video Features & Caption Extraction Completed
Total Number of Loaded Videos = 301
Total Frames Processed = 15050
In [9]:
#normalized image features scaler =
MinMaxScaler((0, 1)) images =
scaler.fit_transform(images)
print("Normalized Features = "+str(images))
Normalized Features = [[0.00000000e+00 3.89156068e-02 3.99136233e-01 ... 4.96405705e01
6.90170467e-01 5.98952162e-01]

```

```
[9.87671316e-04 4.73007520e-04 2.06179054e-03 ... 4.96405699e-01 6.90166287e-01 5.98952145e-01]
```

```
[6.80662453e-01 7.91492926e-03 9.44501732e-03 ... 4.99339982e-01 7.34805465e-01 5.99293903e-01]
```

```
...
```

```
[1.21701939e-03 1.35581595e-03 5.14020451e-03 ... 4.96461098e-01 6.90468989e-01 5.98916891e-01]
```

```
[2.78365216e-03 3.16313955e-03 4.83344811e-03 ... 4.96407647e-01 6.89346075e-01 5.98950005e-01]
```

```
[1.94728468e-03 6.28483176e-02 1.11888196e-01 ... 4.96391051e-01 6.89990946e-01 5.98935253e-01]] In
```

```
[10]:
```

```
#convert captions into keras CNN tokenizer tokenizer =  
Tokenizer() tokenizer.fit_on_texts(captions) vocab_size =  
len(tokenizer.word_index) + 1 max_length =  
max(len(caption.split()) for caption in captions)  
print("Vocabulary Size = "+str(vocab_size)) Vocabulary Size  
= 801 In [11]:
```

```
#function to combine image and caption features to generate train and test array
```

```
def prepare_data(image_features, caption_features): x1, x2, y = [], [], [] for  
m in range(len(image_features)):
```

```
    caption = caption_features[m] caption =  
caption.split() seq =  
tokenizer.texts_to_sequences([caption])[0]  
length = len(seq) for i in range(1, length):  
    x2_seq, y_seq = seq[:i], seq[i] x2_seq = pad_sequences([x2_seq],  
maxlen = max_length, padding='post')[0] y_seq = to_categorical([y_seq],  
num_classes = vocab_size)[0] x1.append(image_features[m])  
x2.append(x2_seq)  
y.append(y_seq)
```

```
    return np.array(x1), np.array(x2), np.array(y) In
```

```
[12]:
```

```
#splitting caption tokens and images into train & test
```

```
train_x1, train_x2, train_y = prepare_data(images[0:250], captions[0:250]) validate_x1,  
validate_x2, validate_y = prepare_data(images[250:len(images)],  
captions[250:len(captions)])
```

```
print("Training Size = "+str(train_x1.shape[0])) print("Validation  
Size = "+str(validate_x1.shape[0]))
```

```
Training Size = 2139
```

```
Validation Size = 409 In
```

```
[13]:
```

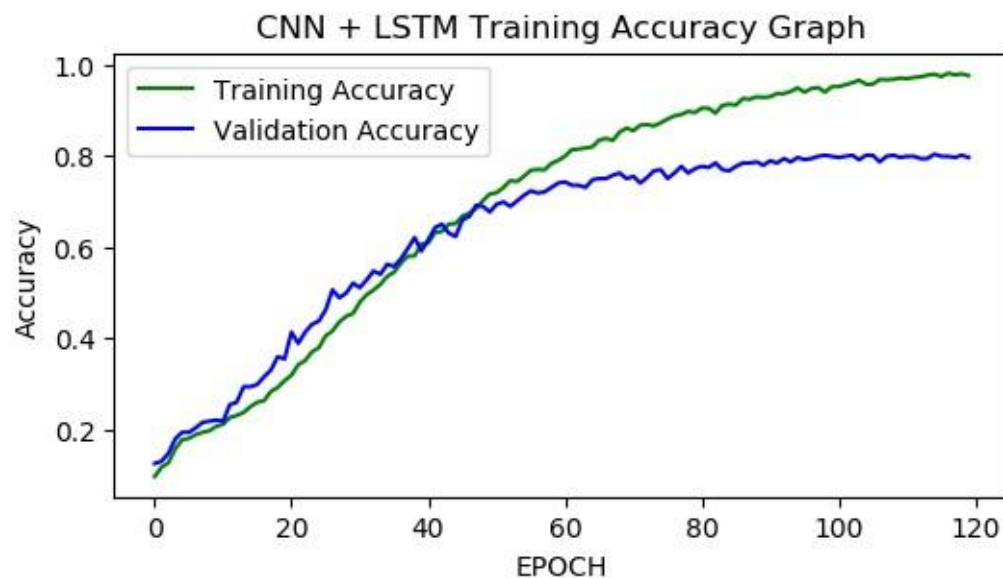
```

#defining CNN & LSTM layers as encoder and decoder model where encoder will encode
images and captions features
#and decoder will decode predicted sentences
inputs1 = Input(shape=(27132,)) fe1 =
Dropout(0.4)(inputs1)
fe2 = Dense(256, activation='relu')(fe1)
# sequence feature layers inputs2 = Input(shape=(max_length,)) se1 = Embedding(vocab_size,
256, mask_zero=True)(inputs2) se2 = Dropout(0.4)(se1) #defining LSTM layer se3 =
LSTM(256)(se2) # decoder model decoder1 = add([fe2, se3]) decoder2 = Dense(256,
activation='relu')(decoder1) outputs = Dense(vocab_size, activation='sigmoid')(decoder2)
cnn_lstm_model = Model(inputs=[inputs1, inputs2], outputs=outputs) optimizer =
Adam(0.001) cnn_lstm_model.compile(loss='categorical_crossentropy', optimizer=optimizer,
metrics =
['accuracy'])
if
os.path.exists("model/cnn_weights.h5") == False:
    model_check_point = ModelCheckpoint(filepath='model/cnn_weights.h5', verbose = 1,
save_best_only = True) hist = cnn_lstm_model.fit([train_x1, train_x2], train_y, verbose = 1,
epochs = 20, callbacks=[model_check_point], validation_data=([validate_x1, validate_x2],
validate_y)) f = open('model/cnn_history.pckl', 'wb') pickle.dump(hist.history, f) f.close()
else:
    cnn_lstm_model.load_weights("model/cnn_weights.h5") #perform
prediction on validation data to calculate loss and accuracy
score = cnn_lstm_model.evaluate([validate_x1, validate_x2], validate_y, verbose=1)
print('Validation loss:', score[0]) print('Validation accuracy:', score[1])
WARNING:tensorflow:From
c:\users\admin\appdata\local\programs\python\python37\lib\sitepackages\tensorflow\python\k
eras\backend.py:3794: add_dispatch_support.<locals>.wrapper (from
tensorflow.python.ops.array_ops) is deprecated and will be removed in a future version.
Instructions for updating:
Use tf.where in 2.0, which has the same broadcast rule as np.where
WARNING:tensorflow:From
c:\users\admin\appdata\local\programs\python\python37\lib\sitepackages\keras\backend\tens
orflow_backend.py:422: The name tf.global_variables is deprecated. Please use
tf.compat.v1.global_variables instead.

409/409 [=====] - 1s 3ms/step
Validation loss: 2.501425291331881
Validation accuracy: 0.7970660328865051 In
[18]:
#visualizing model training and validation accuracy f =
open('model/cnn_history.pckl', 'rb') train_values = pickle.load(f)

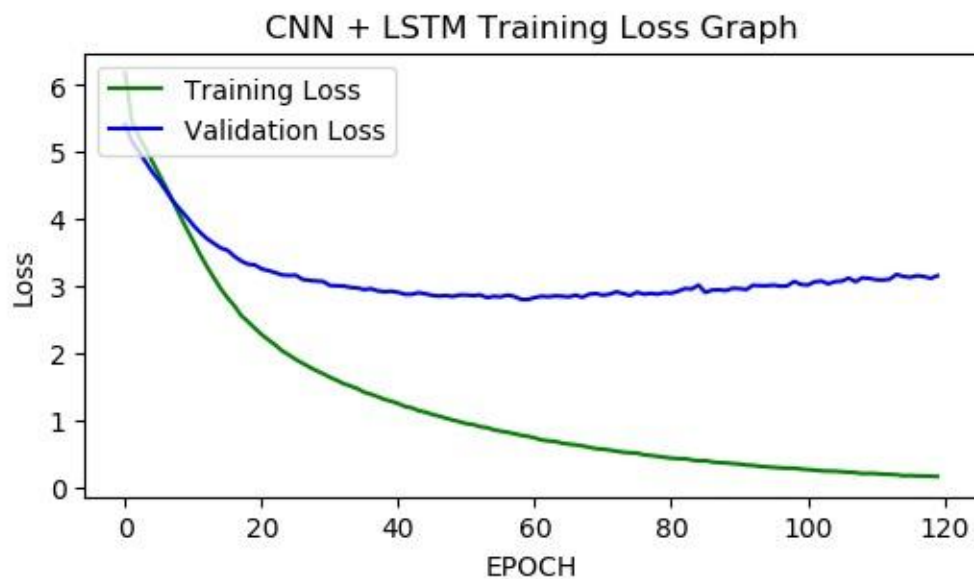
```

```
f.close() plt.figure(figsize=(6, 3)) train_accuracy =
train_values['accuracy'] val_accuracy = train_values['val_accuracy']
plt.xlabel('EPOCH') plt.ylabel('Accuracy') plt.plot(train_accuracy, color
= 'green') plt.plot(val_accuracy, color = 'blue') plt.legend(['Training
Accuracy', 'Validation Accuracy'], loc='upper left') plt.title('CNN +
LSTM Training Accuracy Graph') plt.show()
```



In [19]:

```
#visualizing model training and validation loss
plt.figure(figsize=(6, 3)) train_loss = train_values['loss']
val_loss = train_values['val_loss'] plt.xlabel('EPOCH')
plt.ylabel('Loss') plt.plot(train_loss, color = 'green')
plt.plot(val_loss, color = 'blue') plt.legend(['Training Loss',
'Validation Loss'], loc='upper left') plt.title('CNN + LSTM
Training Loss Graph') plt.show()
```

In [21]:

```
#function to get word by giving tokenizer id def
word_for_id(integer, tokenizr):    for word, index
in tokenizr.word_index.items():    if index ==
integer:
    return word
```

return None In

[23]:

```
#function to predict caption from given video features def
predictCaption(model, tokenizer, features, max_length):
    in_text = 'startseq'    for i in
range(max_length):
        sequence = tokenizer.texts_to_sequences([in_text])[0]    sequence =
pad_sequences([sequence], maxlen=max_length, padding='post')
        yhat = model.predict([features, sequence], verbose=0)
        yhat = np.argmax(yhat)    word =
word_for_id(yhat, tokenizer)    if
word is None:
        break
        in_text += ' ' + word
        # stop if we predict the end of the sequence
if word == 'endseq':
        break    in_text =
in_text.replace('startseq', '')    in_text
= in_text.replace('endseq', '')    return
in_text
```

In [44]:

```

#function to play video and display caption and sentiment def
playVideo(filename, caption, emotion):
    cap = cv2.VideoCapture(filename)
while True:
        ret, frame = cap.read()
if ret == True:
            frame = cv2.resize(frame, (800, 400))
            cv2.putText(frame, caption, (10, 25),
cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 0,
0), 2)
            cv2.putText(frame, "Emotion "+emotion, (10, 55),
cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 0, 0), 2)
            cv2.imshow('Output Video',
frame)
            if cv2.waitKey(5) & 0xFF == ord('q'):
                break
else:
            break
cap.release()
cv2.destroyAllWindows()
In [45]:
def getEmotion(text):
    global analyzer
    scores = analyzer.polarity_scores(text)
    if scores['compound'] >= 0.05:
        return "Positive"
    elif
scores['compound'] <= -0.05:
        return "Negative"
    else:
        return
"Neutral"
In [46]:
#this function read features from given video and predict caption
def readTestVideo(video_test):
    image_features = video_to_frames(video_test)
    photo = []
    photo.append(image_features)
    photo = np.asarray(photo)
    photo =
scaler.transform(photo)
    caption = predictCaption(cnn_lstm_model,
tokenizer, photo, max_length)
    emotion = getEmotion(caption)
    print("Caption = "+caption)
    print("Emotion = "+emotion)
playVideo(video_test, caption, emotion)
In [51]:
#call this function with video file path to detect caption and emotion
readTestVideo("testVideos/1.avi")
Caption = bird in sink keeps getting under the running water from faucet
Emotion = Neutral
In [52]:
#call this function with video file path to detect caption and emotion
readTestVideo("testVideos/2.avi")

```

Caption = man playing guitar outside his house

Emotion = Positive In [53]:

#call this function with video file path to detect caption and emotion

readTestVideo("testVideos/3.avi")

Caption = woman cuts an onion

Emotion = Negative In

[50]:

#call this function with video file path to detect caption and emotion

readTestVideo("testVideos/4.avi")

Caption = woman is putting make up on her face

Emotion = Neutral

In []:

REFERENCES:

- Cheng, W., Yang, H., Bartz, C., & Meinel, C. (2016). Image captioning with deep bidirectional LSTMs. arXiv preprint arXiv:1604.00790.
arXiv
- Bin, Y., Yang, Y., Huang, Z., Shen, F., Xu, X., & Shen, H. T. (2016). Bidirectional long-short term memory for video description. arXiv preprint arXiv:1606.04631.
arXiv
- Guo, Y., Zhang, J., & Gao, L. (2022). Exploiting long-term temporal dynamics for video captioning. arXiv preprint arXiv:2202.10828.
arXiv
- Xu, K., Ba, J., Kiros, J., Cho, K., Courville, A., Salakhutdinov, R., ... & Bengio, Y. (2015). Show, attend and tell: Neural image caption generation with visual attention. Proceedings of the 32nd International Conference on Machine Learning (ICML), 2048–2057.
- Vinyals, O., Toshev, A., Bengio, S., & Erhan, D. (2015). Show and tell: A neural image caption generator. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 3156–3164.
- Donahue, J., Hendricks, L. A., Rohrbach, M., Venugopalan, S., Saenko, K., & Darrell, T. (2015). Long-term recurrent convolutional networks for visual recognition and description. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2625–2634.
- Pan, Y., & Yang, Q. (2016). Hierarchical LSTM with attention for image captioning. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 3358–3366.
- Lu, J., Yang, J., Batra, D., & Parikh, D. (2017). Visual semantic role labeling: A benchmark for image understanding. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 1209–1218.
- Huang, Z., & Yu, H. (2019). Attention on attention: A novel approach to visual question answering. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 10013–10022.
- Chen, X., & Zitnick, C. L. (2015). Mind’s eye: A recurrent visual representation for image caption generation. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2422–2431.

Emotion Recognition with CNN+LSTM

Mustaqeem, M., & Kwon, S. (2020). CLSTM: Deep feature-based speech emotion recognition using the hierarchical ConvLSTM network. *Mathematics*, 8(12), 2133.

MDPI

Etienne, C., Fidanza, G., Petrovskii, A., Devillers, L., & Schmauch, B. (2018). CNN+LSTM architecture for speech emotion recognition with data augmentation. *arXiv preprint arXiv:1802.05630*.

arXiv

Zhang, L., Wang, L., Dang, J., Guo, L., & Yu, Q. (2018). Gender-aware CNN-BLSTM for speech emotion recognition. *Artificial Neural Networks and Machine Learning – ICANN 2018*. Springer, Cham.

SpringerLink

Wang, J., Xue, M., Culhane, R., Diao, E., Ding, J., & Tarokh, V. (2020). Speech emotion recognition with dual-sequence LSTM architecture. *IEEE ICASSP 2020*.

IEEE Resource Center

Peng, Z., Lu, Y., Pan, S., & Liu, Y. (2021). Efficient speech emotion recognition using multiscale CNN and attention. *IEEE ICASSP 2021*.

IEEE Resource Center

Chakravarthi, B., Ng, S.-C., Ezilarasan, M. R., & Leung, M.-F. (2022). EEG-based emotion recognition using hybrid CNN and LSTM classification. *Frontiers in Computational Neuroscience*, 16, 1019776.

Frontiers

Huang, Z., Ma, Y., Wang, R., Li, W., & Dai, Y. (2023). A model for EEG-based emotion recognition: CNN-Bi-LSTM with attention mechanism. *Electronics*, 12(14), 3188.

MDPI

Zhang, Z., & Zhang, Y. (2019). Speech emotion recognition using CNN and LSTM. *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 7225–7229.

Li, X., & Liu, Y. (2020). Multi-modal emotion recognition using CNN and LSTM. *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 7220–7224.